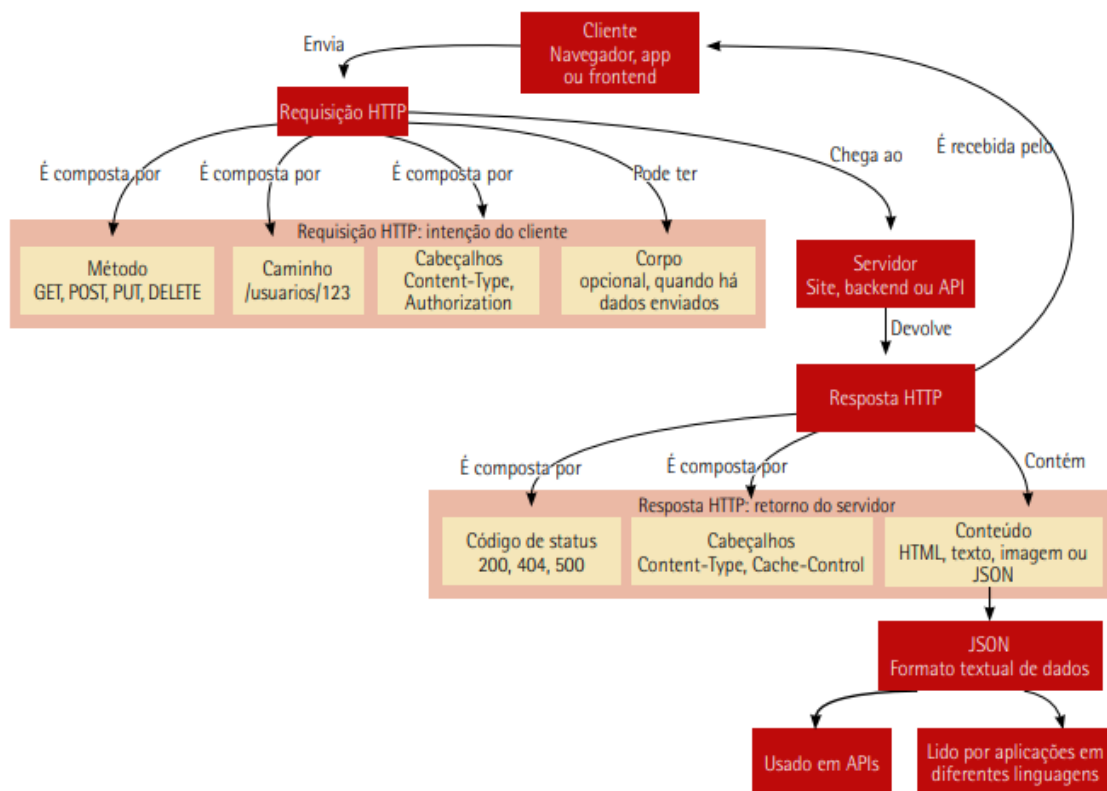


1 PIPELINE HTTP E ENDPOINTS.....	2
1.1 Middleware, roteamento e binding de parâmetros.....	5
1.2 Minimal APIs e MVC: critérios de escolha.....	14
2 MVC E RAZOR.....	18
2.1 Controllers, validação (Data Annotations) e filtros.....	19
2.2 Views, layouts/partials e Tag Helpers.....	28
3 PERSISTÊNCIA E CONEXÃO COM BD COM EF CORE.....	31
3.1 Modelagem, DbContext, migrations, seed e pooling de conexões.....	32
3.2 Transações/concorrência (RowVersion, DbTransaction), leituras e comandos eficientes.....	41
4 APIS REST E OPENAPI.....	50
4.1 Recursos, paginação/ordenação/filtros e erros (Problem Details).....	51
4.2 Documentação com Swagger e geração de cliente TypeScript (NSwag).....	55
5 INTEGRAÇÃO REACT/NODE – CONSUMO DA API.....	57
5.1 Setup do front-end (Node LTS, Vite/Next), CORS e proxy de desenvolvimento.....	58
5.2 Fetch/axios, estados de requisição, roteamento e páginas de lista/detalhe.....	66
6 AUTENTICAÇÃO E AUTORIZAÇÃO PONTA A PONTA.....	71
6.1 ASP.NET Identity, cookies × JWT e roles/claims.....	72
6.2 SPA – login/logout, armazenamento seguro de tokens e rotas protegidas.....	79
7 OBSERVABILIDADE E RESILIÊNCIA.....	88
7.1 Logging, métricas, tracing e Health Checks.....	89
8 BUILD, IMPLANTAÇÃO E CI/CD FULL-STACK.....	99
8.1 Configuração por ambientes, secrets e publicação containerizada.....	100
8.2 Deploy integrado (API servindo estáticos do React) ou pipelines separados (API + front-end).....	106

Unidade I

1 PIPELINE HTTP E ENDPOINTS

Toda aplicação web nasce de uma relação fundamental entre cliente e servidor. Um navegador, um aplicativo móvel ou outro sistema envia uma requisição solicitando algum recurso; a aplicação recebe essa requisição, interpreta seu propósito, executa as operações necessárias e devolve uma resposta. Em ASP.NET Core, o caminho percorrido pela requisição desde sua chegada ao servidor até a produção da resposta é chamado de pipeline HTTP.



O pipeline HTTP pode ser entendido como uma sequência organizada de etapas pelas quais cada requisição passa. Essas etapas definem como a aplicação decide o que fazer diante de uma URL (Uniform Resource Locator, em português, Localizador Uniforme de Recursos), como interpreta informações enviadas pelo usuário, como aplica regras de segurança, como escolhe o código responsável pelo atendimento da solicitação e como transforma o resultado desse processamento em uma resposta compreensível para o cliente.

Para ilustrar com uma aplicação típica, criaremos o PetMatch, cujo propósito é servir como base para um sistema de doação e adoção responsável de pets, no qual pessoas, organizações e interessados em adoção possam interagir por meio de

recursos digitais organizados. Nele, uma pessoa interessada em adotar um pet poderá acessar uma página com animais disponíveis, filtrar informações, abrir detalhes de um cadastro, enviar uma proposta de adoção ou consultar orientações sobre posse responsável. Cada uma dessas ações começa como uma requisição HTTP, mas cada requisição pode exigir decisões distintas da aplicação. Algumas apenas consultam informações, outras enviam dados, outras dependem de validação, autenticação, roteamento adequado e produção de uma resposta visual ou estruturada. O pipeline é o mecanismo que permite organizar esse fluxo de maneira coerente.

Em ASP.NET Core, middleware é um componente que participa do processamento da requisição e da resposta. Ele pode examinar a requisição, modificá-la, decidir se deve encaminhá-la para a próxima etapa ou interromper o fluxo produzindo uma resposta diretamente, não representa uma tela nem uma regra de negócio específica; ele compõe o ambiente pelo qual as solicitações circulam.

O roteamento assume papel decisivo. Quando uma requisição chega com determinado endereço, a aplicação precisa identificar qual parte do código é responsável por atendê-la.

O roteamento estabelece essa associação entre padrões de URL e pontos de execução. Em vez de espalhar decisões manuais sobre endereços ao longo do sistema, o ASP.NET Core oferece um modelo no qual as rotas descrevem caminhos possíveis e os endpoints representam os destinos capazes de produzir respostas. Um endpoint é, nesse sentido, o ponto final lógico do percurso da requisição dentro da aplicação.

Quadro 1

Elemento	Exemplo no PetMatch	Papel
Método HTTP	GET	Indica a intenção geral da requisição, como consultar dados
URL acessada	<code>https://localhost:porta/api/pets/2</code>	Endereço completo usado pelo navegador ou cliente HTTP
Caminho	<code>/api/pets/2</code>	Parte da URL analisada pelo roteamento da aplicação
Rota	<code>/api/pets/{id:int}</code>	Padrão declarado no código para reconhecer caminhos semelhantes
Endpoint	<code>MapGet("/api/pets/{id:int}", ...)</code>	Associação entre método, rota e código que produzirá a resposta

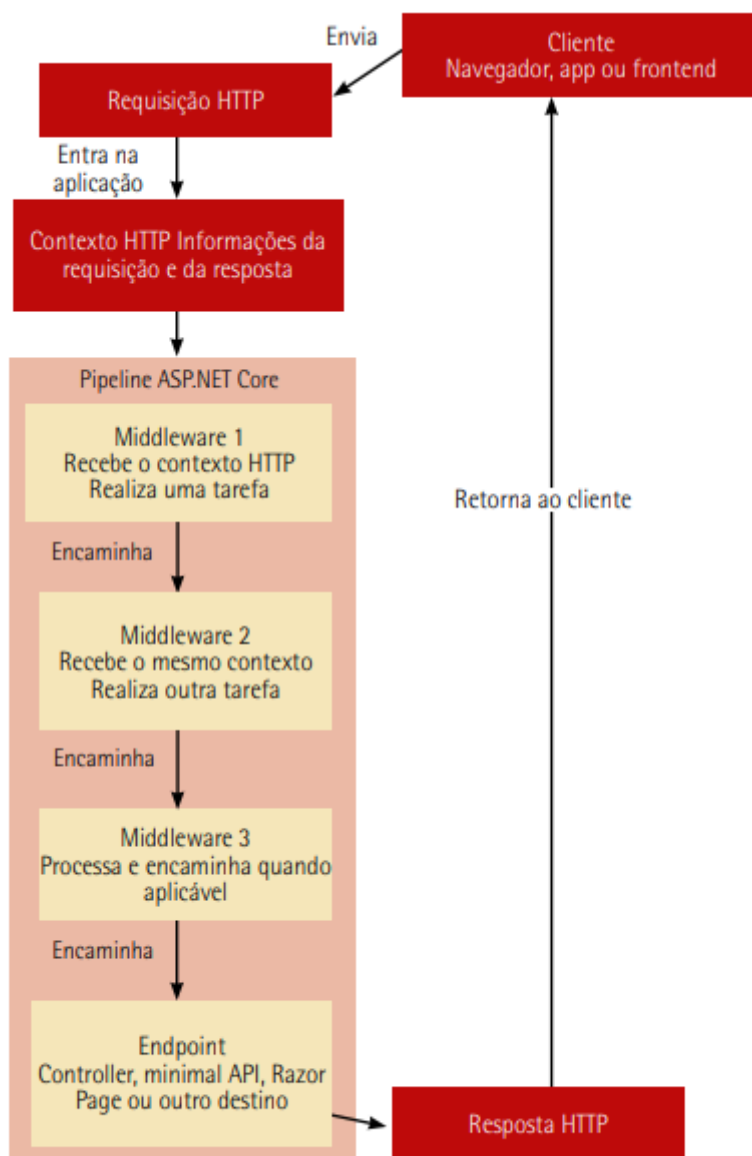


Figura 2 – Pipeline: middlewares e endpoint

O binding, isto é, o mecanismo pelo qual dados presentes na requisição são associados a parâmetros, propriedades ou objetos usados pelo código da aplicação. Quando um usuário envia informações por formulário, consulta uma página com filtros na URL ou acessa um recurso identificado por um valor presente na rota, esses dados precisam ser convertidos em valores manipuláveis pelo programa. O binding reduz a distância entre o protocolo HTTP, que trafega textos, cabeçalhos, caminhos e corpos de mensagem, e a linguagem C#, que trabalha com tipos, objetos e estruturas fortemente definidas.

O middleware prepara e conduz o ambiente de execução; o roteamento identifica o destino adequado; o endpoint executa a ação correspondente; o binding transforma dados da requisição em valores utilizáveis

1.1 Middleware, roteamento e binding de parâmetros

Pipeline HTTP, ela conduz cada solicitação por uma sequência definida de responsabilidades até encontrar o endpoint adequado.

O middleware é o componente que participa desse percurso. Cada middleware ocupa uma posição na cadeia de processamento e pode executar uma tarefa específica sobre a requisição ou a resposta. Alguns middlewares preparam a aplicação para servir arquivos estáticos, outros redirecionam o tráfego para HTTPS, outros podem autenticar usuários, autorizar acessos, registrar eventos ou tratar erros.

No PetMatch, mesmo a primeira versão da aplicação já precisa desse fluxo. A página inicial (figura 3) possui um arquivo estático, com HTML (HyperText Markup Language ou Linguagem de Marcação de Hipertexto), CSS (Cascading Style Sheets ou Folha de Estilo em Cascata) e JavaScript. Ao mesmo tempo, a aplicação disponibiliza endpoints que retornam dados em JSON (JavaScript Object Notation ou Notação de Objetos JavaScript) para preencher os cards de pets, consultar um animal pelo identificador, receber filtros por query string (veremos em breve) e registrar uma manifestação de interesse enviada pelo formulário. Isso significa que algumas requisições serão atendidas pelo middleware de arquivos estáticos, enquanto outras serão encaminhadas aos endpoints da API.

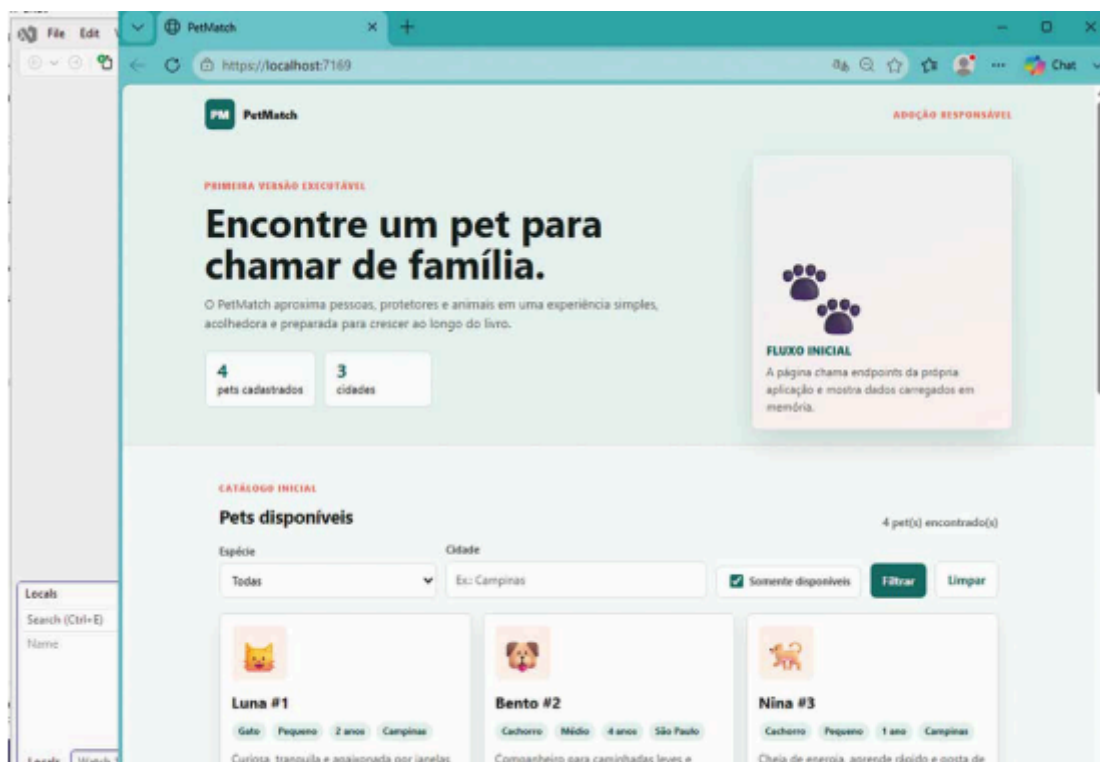


Figura 3 – Página inicial do PetMatch

Quando o navegador solicita `/api/pets`, a aplicação precisa reconhecer que essa rota representa a listagem de pets disponíveis. Quando a solicitação aponta para `/api/pets/2`, o trecho numérico da URL deve ser interpretado como o identificador de um pet. Quando o endereço contém filtros, como espécie, porte ou cidade, a aplicação deve associar esses valores aos parâmetros esperados pelo código.

O endpoint é o destino executável da requisição. Em Minimal APIs (estudaremos adiante), um endpoint pode ser declarado diretamente em `Program.cs`, por meio de métodos como `MapGet` e `MapPost`.

O binding de parâmetros completa esse raciocínio ao mostrar como dados HTTP se tornam valores C#. O protocolo HTTP transporta informações em partes diferentes da requisição. Um valor pode estar no caminho da rota, como o identificador em `/api/pets/2`; pode estar na query string, como `?especie=Cachorro`; ou pode estar no corpo da requisição, como ocorre quando um formulário envia dados em JSON. O ASP.NET Core consegue associar esses dados a parâmetros de métodos e objetos tipados, reduzindo a necessidade de manipular manualmente a requisição bruta.

```
namespace PetMatch.Web.Models;
```

```
public sealed record Pet(  
    int Id,  
    string Nome,  
    string Espécie,  
    string Porte,  
    int IdadeAproximada,  
    string Cidade,  
    string Descricao,  
    string Icone,  
    bool Disponivel,  
    string[] Caracteristicas);
```

```
namespace PetMatch.Web.Models;
```

```
public sealed record AdoptionInterestRequest(  
    int PetId,  
    string Nome,  
    string Email,  
    string Mensagem);
```

```

using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

public sealed class PetMemoryStore
{
    private readonly List<Pet> _pets =
    [
        new Pet(
            1,
            "Luna",
            "Cachorro",
            "Médio",
            3,
            "Curitiba",
            "Dócil, brincalhona e acostumada com crianças.",
            "🐶",
            true,
            ["Vacinada", "Castrada", "Sociável"]),

        new Pet(
            2,
            "Mingau",
            "Gato",
            "Pequeno",
            2,
            "São Paulo",
            "Calmo, curioso e ideal para apartamento.",
            "🐱",
            true,
            ["Vermifugado", "Independente", "Carinhoso"]),

        new Pet(
            3,
            "Thor",
            "Cachorro",
            "Grande",
            5,
            "Belo Horizonte",
            "Protetor, ativo e indicado para lares com espaço.",
            "🐶",
            true,
            ["Adestrado", "Energético", "Companheiro"]),

        new Pet(
            4,
            "Amora",
            "Gato",
            "Pequeno",
            1,
            "Florianópolis",
            "Filhote afetuosa, resgatada recentemente.",
            "🐱",
            true,
            []
        )
    ];
}

```

```

        ["Filhote", "Brincalhona", "Resgatada"])
    };

    public IReadOnlyList<Pet> Listar(string? especie, string? porte, string?
cidade)
    {
        IEnumerable<Pet> consulta = _pets.Where(pet => pet.Disponivel);

        if (!string.IsNullOrEmpty(especie))
        {
            consulta = consulta.Where(pet =>
                pet.Especie.Equals(especie, StringComparison.OrdinalIgnoreCase));
        }

        if (!string.IsNullOrEmpty(porte))
        {
            consulta = consulta.Where(pet =>
                pet.Porte.Equals(porte, StringComparison.OrdinalIgnoreCase));
        }

        if (!string.IsNullOrEmpty(cidade))
        {
            consulta = consulta.Where(pet =>
                pet.Cidade.Contains(cidade, StringComparison.OrdinalIgnoreCase));
        }

        return consulta.ToList();
    }

    public Pet? ObterPorId(int id)
    {
        return _pets.FirstOrDefault(pet => pet.Id == id);
    }
}

```

Requisição	Onde o dado está no HTTP	Parâmetro C# preenchido
GET /api/pets?especie=Gato	Query string	string? especie
GET /api/pets/2	Rota	int id
GET /api/boas-vindas/Ana	Rota	string nome
POST /api/interesses com JSON no corpo	Corpo da requisição	AdoptionInterestRequest request

```

using PetMatch.Web.Data;
using PetMatch.Web.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<PetMemoryStore>();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapGet("/api/pets", (
    PetMemoryStore store,
    string? especie,
    string? porte,
    string? cidade) =>
{
    var pets = store.Listar(especie, porte, cidade);
    return Results.Ok(pets);
});

app.MapGet("/api/pets/{id:int}", (PetMemoryStore store, int id) =>
{
    var pet = store.ObterPorId(id);

    return pet is null
        ? Results.NotFound(new { mensagem = "Pet não encontrado." })
        : Results.Ok(pet);
});

app.MapGet("/api/boas-vindas/{nome}", (string nome) =>
{

```

```

var nomeTratado = string.IsNullOrEmpty(nome)
    ? "visitante"
    : nome.Trim();

return Results.Ok(new
{
    mensagem = $"Olá, {nomeTratado}! O PetMatch ajuda você a encontrar uma
adoção responsável."
});
});

app.MapPost("/api/interesses", (
    PetMemoryStore store,
    AdoptionInterestRequest request) =>
{
    if (request.PetId <= 0 ||
        string.IsNullOrEmpty(request.Nome) ||
        string.IsNullOrEmpty(request.Email) ||
        !request.Email.Contains('@') ||
        string.IsNullOrEmpty(request.Mensagem))
    {
        return Results.BadRequest(new
        {
            mensagem = "Preencha nome, e-mail, mensagem e um pet válido."
        });
    }

    var pet = store.ObterPorId(request.PetId);

    if (pet is null)
    {
        return Results.NotFound(new
        {
            mensagem = "Não foi possível localizar o pet informado."
        });
    }

    return Results.Ok(new
    {
        mensagem = $"Interesse registrado para {pet.Nome}. A equipe responsável
poderá avaliar o contato.",
        pet = pet.Nome,
        interessado = request.Nome.Trim()
    });
});

app.Run();

```

Antes de observar os endpoints, é importante entender a linha `builder.Services.AddSingleton()`. Ela registra `PetMemoryStore` no contêiner de serviços da aplicação. Esse contêiner é responsável por criar e fornecer objetos que os endpoints precisam usar. Por isso, quando um endpoint declara o parâmetro `PetMemoryStore store`, o ASP.NET Core entende que esse valor não veio da rota, da query string nem do corpo JSON; ele deve ser obtido dos serviços registrados. O uso de `Singleton` significa que a mesma instância de `PetMemoryStore` será reutilizada enquanto a aplicação estiver em execução, o que é adequado aqui porque os dados estão apenas em memória e servem como base didática.

A ordem dos middlewares nesse arquivo é significativa. UseHttpsRedirection orienta a aplicação a trabalhar com HTTPS quando o perfil seguro estiver em uso. UseDefaultFiles permite que a requisição à raiz da aplicação encontre automaticamente o arquivo inicial dentro de wwwroot. UseStaticFiles entrega HTML, CSS e JavaScript ao navegador.

A ligação entre a página e a API é feita pelo arquivo wwwroot/js/petmatch.js, no projeto PetMatch.Web. Esse script usa fetch para chamar os endpoints, demonstrando como as respostas JSON produzidas pelo ASP.NET Core alimentam a interface.

```
const listaPets = document.getElementById("listaPets");
const totalPets = document.getElementById("totalPets");
const filtroEspecie = document.getElementById("filtroEspecie");
const filtroPorte = document.getElementById("filtroPorte");
const filtroCidade = document.getElementById("filtroCidade");
const botaoFiltrar = document.getElementById("botaoFiltrar");
const petIdConsulta = document.getElementById("petIdConsulta");
const botaoConsultar = document.getElementById("botaoConsultar");
const resultadoConsulta = document.getElementById("resultadoConsulta");
const petInteresse = document.getElementById("petInteresse");
const formInteresse = document.getElementById("formInteresse");
const retornoInteresse = document.getElementById("retornoInteresse");

async function carregarPets() {
    const parametros = new URLSearchParams();

    if (filtroEspecie.value) {
        parametros.set("especie", filtroEspecie.value);
    }

    if (filtroPorte.value) {
        parametros.set("porte", filtroPorte.value);
    }

    if (filtroCidade.value.trim()) {
        parametros.set("cidade", filtroCidade.value.trim());
    }

    const resposta = await fetch(`/api/pets?${parametros.toString()}`);
    const pets = await resposta.json();

    totalPets.textContent = `${pets.length} pet${pets.length === 1 ? "" : "s"}`;
    preencherCards(pets);
    preencherSelecaoDeInteresse(pets);
}

function preencherCards(pets) {
    listaPets.innerHTML = "";

    if (pets.length === 0) {
        listaPets.innerHTML = "<p>Nenhum pet encontrado com os filtros informados.</p>";
        return;
    }

    for (const pet of pets) {
        const card = document.createElement("article");
        card.className = "pet-card";
    }
}
```

```

card.innerHTML = `
  <span class="icon">${pet.icone}</span>
  <h3>${pet.nome}</h3>
  <p>${pet.descricao}</p>
  <div class="badges">
    <span class="badge">${pet.especie}</span>
    <span class="badge">${pet.porte}</span>
    <span class="badge">${pet.idadeAproximada} ano(s)</span>
    <span class="badge">${pet.cidade}</span>
  </div>
  <small>${pet.caracteristicas.join(" • ")}</small>
`;

listaPets.appendChild(card);
}
}

function preencherSelecaoDeInteresse(pets) {
  petInteresse.innerHTML = "";

  for (const pet of pets) {
    const opcao = document.createElement("option");
    opcao.value = pet.id;
    opcao.textContent = `${pet.nome} - ${pet.especie}`;
    petInteresse.appendChild(opcao);
  }
}

async function consultarPetPorId() {
  const id = petIdConsulta.value;

  const resposta = await fetch(`/api/pets/${id}`);
  const conteudo = await resposta.json();

  if (!resposta.ok) {
    resultadoConsulta.textContent = conteudo.mensagem;
    return;
  }

  resultadoConsulta.innerHTML = `
    <strong>${conteudo.nome}</strong><br>
    ${conteudo.especie}, porte ${conteudo.porte}, em ${conteudo.cidade}.<br>
    ${conteudo.descricao}
  `;
}

```

```

}

async function enviarInteresse(evento) {
  evento.preventDefault();

  const corpo = {
    petId: Number(petInteresse.value),
    nome: document.getElementById("nomeInteresse").value,
    email: document.getElementById("emailInteresse").value,
    mensagem: document.getElementById("mensagemInteresse").value
  };

```

Unidade I

```

const resposta = await fetch("/api/interesses", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(corpo)
});

const conteudo = await resposta.json();
retornoInteresse.textContent = conteudo.mensagem;

if (resposta.ok) {
  formInteresse.reset();
}

}

botaoFiltrar.addEventListener("click", carregarPets);
botaoConsultar.addEventListener("click", consultarPetPorId);
formInteresse.addEventListener("submit", enviarInteresse);

carregarPets();

```

1.2 Minimal APIs e MVC: critérios de escolha

As Minimal APIs são uma forma mais direta e concisa de definir endpoints HTTP, isto é, pontos de entrada da aplicação associados a rotas como GET, POST, PUT ou DELETE. Nesse modelo, a rota, os parâmetros recebidos, a lógica executada e a resposta produzida tendem a aparecer próximos no código, normalmente em declarações simples no arquivo de configuração da aplicação.

O padrão MVC organiza a aplicação a partir da separação entre Model, View e Controller. O Model representa os dados e as regras de domínio ou de negócio; a View corresponde à camada responsável pela apresentação das informações ao

usuário, quando há interface visual renderizada pelo servidor; e o Controller atua como intermediário, recebendo as requisições HTTP, coordenando o processamento necessário e escolhendo a resposta adequada. Dentro de um controller, as actions são métodos específicos que representam operações acessíveis por determinadas rotas, como listar registros, consultar um item, criar um recurso ou atualizar uma informação existente.

No PetMatch, a aplicação ainda possui um escopo adequado para Minimal APIs. Ela entrega uma página inicial estática, visualmente estruturada, e expõe endpoints para listar pets, consultar um pet por identificador, retornar uma mensagem de boas-vindas e receber uma manifestação de interesse. Esses fluxos são suficientes para demonstrar uma aplicação real, mas ainda não exigem a estrutura formal de controllers e views. A consequência prática é importante: a aplicação pode continuar simples sem se tornar desorganizada, desde que o código seja progressivamente extraído para arquivos com responsabilidade clara.

No PetMatch, a criação da pasta Endpoints e do arquivo PetEndpoints.cs demonstra essa possibilidade. A aplicação continua com os mesmos caminhos públicos e com o mesmo comportamento no navegador, mas a estrutura interna passa a indicar melhor onde vivem as rotas relacionadas aos pets e às manifestações de interesse.

A estrutura usada preserva os arquivos criados anteriormente e acrescenta apenas uma pasta para os endpoints. No projeto `PetMatch.Web`, a organização relevante passa a ser a seguinte:

```
src/  
  PetMatch.Web/
```

Unidade I

```
Program.cs  
Data/  
  PetMemoryStore.cs  
Endpoints/  
  PetEndpoints.cs  
Models/  
  AdoptionInterestRequest.cs  
  Pet.cs  
wwwroot/  
  index.html  
  css/  
    petmatch.css  
  js/  
    petmatch.js  
Properties/  
  launchSettings.json
```

No projeto PetMatch.Web, o arquivo Program.cs passa a ter o papel de configurar os serviços, montar a aplicação, registrar os middlewares e chamar o método que mapeia os endpoints.

```
using PetMatch.Web.Data;
using PetMatch.Web.Endpoints;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<PetMemoryStore>();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();
```

6

DESENVOLVIMENTO WEB COM .NET

```
app.MapPetEndpoints();

app.Run();
```

```
using PetMatch.Web.Data;
using PetMatch.Web.Models;

namespace PetMatch.Web.Endpoints;

public static class PetEndpoints
{
    public static WebApplication MapPetEndpoints(this WebApplication app)
    {
        var api = app.MapGroup("/api");

        api.MapGet("/pets", (
            PetMemoryStore store,
            string? especie,
            string? porte,
            string? cidade) =>
        {
            var pets = store.Listar(especie, porte, cidade);
            return Results.Ok(pets);
        });

        api.MapGet("/pets/{id:int}", (PetMemoryStore store, int id) =>
        {
            var pet = store.ObterPorId(id);

            return pet is null
                ? Results.NotFound(new { mensagem = "Pet não encontrado." })
                : Results.Ok(pet);
        });

        api.MapGet("/boas-vindas/{nome}", (string nome) =>
        {
            var nomeTratado = string.IsNullOrEmpty(nome)
                ? "visitante"
                : nome.Trim();

            return Results.Ok(new
            {

```

```

        mensagem = $"Olá, {nomeTratado}! O PetMatch ajuda você a
encontrar uma adoção responsável."
    });

    });

    api.MapPost("/interesses", (
        PetMemoryStore store,
        AdoptionInterestRequest request) =>
    {
        if (request.PetId <= 0 ||
            string.IsNullOrEmpty(request.Nome) ||
            string.IsNullOrEmpty(request.Email) ||
            !request.Email.Contains('@') ||
            string.IsNullOrEmpty(request.Mensagem))
        {
            return Results.BadRequest(new
            {
                mensagem = "Preencha nome, e-mail, mensagem e um pet
válido."
            });
        }

        var pet = store.ObterPorId(request.PetId);
        if (pet is null)
        {
            return Results.NotFound(new
            {
                mensagem = "Não foi possível localizar o pet informado."
            });
        }

        return Results.Ok(new
        {
            mensagem = $"Interesse registrado para {pet.Nome}. A equipe
responsável poderá avaliar o contato.",
            pet = pet.Nome,
            interessado = request.Nome.Trim()
        });
    });

    return app;
}
}

```

2 MVC E RAZOR

Um sistema orientado a páginas precisa organizar a entrada de dados, a interpretação das ações do usuário, a aplicação de regras de validação, a seleção das informações que serão exibidas e a composição da interface visual. Sem uma estrutura clara para distribuir essas responsabilidades, a aplicação tende a misturar lógica de navegação, regras de negócio e marcação HTML em trechos difíceis de compreender, testar e manter.

MVC é a abreviação de Model, View e Controller. O modelo representa dados e conceitos com os quais a aplicação trabalha, a visão é responsável pela apresentação visual entregue ao usuário e o controlador atua como elemento intermediário, recebendo a solicitação, coordenando o processamento necessário e escolhendo qual resposta deve ser produzida.

Razor é a tecnologia que permite construir essas visões no ASP.NET Core combinando HTML com expressões C# de forma controlada. Sua finalidade não é substituir o HTML, mas enriquecê-lo com recursos dinâmicos necessários à geração de páginas. Por meio de Razor, uma visão pode exibir valores vindos do modelo, condicionar a apresentação de determinados elementos, percorrer coleções de dados e integrar mensagens de validação ao conteúdo visual.

A relação entre MVC e Razor deve ser entendida como complementar. O MVC define a separação das responsabilidades fundamentais da aplicação orientada a páginas e Razor oferece a linguagem de marcação dinâmica usada principalmente na camada de apresentação.

2.1 Controllers, validação (Data Annotations) e filtros

O projeto PetMatch.Web continua servindo a página estática inicial e os endpoints `/api/pets`, `/api/pets/{id:int}`, `/api/boas-vindas/{nome}` e `/api/interesses`. A evolução consiste em acrescentar controllers, views Razor, ViewModels, validação declarativa e um filtro simples. Com isso, a aplicação passa a ter duas superfícies: uma API em Minimal APIs e uma área MVC acessível por `/Pets`. Essa convivência mostra que ASP.NET Core permite compor estilos diferentes dentro do mesmo projeto quando cada um resolve uma parte clara do problema.

A convivência entre Minimal APIs e MVC pode ser entendida como duas portas de entrada para a mesma aplicação. Os endpoints `/api/pets` continuam retornando dados em JSON, úteis para JavaScript, outras aplicações ou testes diretos no navegador. Já `/Pets` passa a representar uma página MVC completa, com controller, view Razor, layout e formulário.

Quadro 3

Caminho	Estilo	Código acionado	Resposta
/api/pets	Minimal API	MapGet em PetEndpoints	JSON
/api/pets/1	Minimal API	Endpoint de consulta por id	JSON
/Pets	MVC	PetsController.Index()	Página HTML
/Pets/Create	MVC	PetsController.Create()	Formulário HTML

```
src/  
  PetMatch.Web/  
    Controllers/  
      PetsController.cs  
    Data/  
      PetMemoryStore.cs  
    Endpoints/  
      PetEndpoints.cs  
    Filters/  
      PetMatchActionFilter.cs  
    Models/  
      AdoptionInterestRequest.cs  
      Pet.cs  
    ViewModels/  
      PetCreateViewModel.cs  
    Views/  
      _ViewImports.cshtml  
      _ViewStart.cshtml  
      Pets/  
        Index.cshtml  
        Create.cshtml  
      Shared/  
        _Layout.cshtml  
  wwwroot/  
    index.html  
    css/  
      petmatch.css  
      petmatch-mvc.css  
    js/  
      petmatch.js
```

Controllers recebe as classes que coordenam requisições e escolhem respostas. ViewModels recebe objetos voltados a telas e formulários. Views contém as páginas Razor entregues ao navegador. Views/Shared concentra elementos reaproveitados por mais de uma view, como layout e componentes visuais. Filters reúne extensões do fluxo MVC. A pasta Data continua guardando o armazenamento em memória, compartilhado pela API e pela área MVC. Assim, a nova estrutura não substitui a anterior, mas organiza a parte visual e orientada a páginas que está sendo adicionada ao PetMatch.

É importante distinguir o modelo de domínio do ViewModel. O Pet representa um pet dentro da aplicação: é o dado que a listagem, a API e o armazenamento em memória manipulam. Já PetCreateViewModel representa o formulário de cadastro: ele descreve aquilo que o usuário pode preencher na tela e quais regras imediatas esses campos devem obedecer. Por isso, o Pet foi definido como record, adequado para representar um conjunto de dados já formado, enquanto o ViewModel é uma class com propriedades get; set;, pois o model binding do MVC precisa criar o objeto e preencher suas propriedades a partir dos campos enviados pelo formulário. No projeto PetMatch. Web, o arquivo ViewModels/PetCreateViewModel.cs contém esses campos e os atributos de Data Annotations que expressam as regras de preenchimento.

```

using System.ComponentModel.DataAnnotations;

namespace PetMatch.Web.ViewModels;

public sealed class PetCreateViewModel
{
    [Required(ErrorMessage = "Informe o nome do pet.")]
    [StringLength(60, MinimumLength = 2, ErrorMessage = "O nome deve ter entre
2 e 60 caracteres.")]
    [Display(Name = "Nome do pet")]
    public string Nome { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe a espécie.")]
    [StringLength(30, ErrorMessage = "A espécie deve ter no máximo
30 caracteres.")]
    public string Especie { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe o porte.")]
    [StringLength(30, ErrorMessage = "O porte deve ter no máximo
30 caracteres.")]
    public string Porte { get; set; } = string.Empty;

    [Range(0, 30, ErrorMessage = "A idade aproximada deve estar entre 0 e
30 anos.")]
    [Display(Name = "Idade aproximada")]
    public int IdadeAproximada { get; set; }

    [Required(ErrorMessage = "Informe a cidade.")]
    [StringLength(80, ErrorMessage = "A cidade deve ter no máximo
80 caracteres.")]
    public string Cidade { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe uma descrição.")]
    [StringLength(220, MinimumLength = 20, ErrorMessage = "A descrição deve ter
entre 20 e 220 caracteres.")]
    public string Descricao { get; set; } = string.Empty;

    [Required(ErrorMessage = "Informe um ícone para o card.")]
    [StringLength(8, ErrorMessage = "Use um ícone curto para representar o
pet.")]
    [Display(Name = "Ícone")]
    public string Icone { get; set; } = "🐾";

    [Required(ErrorMessage = "Informe ao menos uma característica.")]
    [StringLength(160, ErrorMessage = "As características devem ter no máximo
160 caracteres.")]
    public string Caracteristicas { get; set; } = string.Empty;
}

```

Esse ViewModel demonstra que a validação fica declarada junto ao contrato de entrada do formulário. Quando o usuário envia dados para a action, o model binding preenche uma instância de PetCreateViewModel e o MVC avalia os atributos antes que a action decida como responder.

O armazenamento em memória continua sendo a fonte de dados da aplicação. Para permitir que o formulário MVC cadastre um novo pet, o arquivo Data/PetMemoryStore.cs, no projeto PetMatch.Web, recebe um método Add.

```
using PetMatch.Web.ViewModels;

public Pet Add(PetCreateViewModel viewModel)
{
    var id = _pets.Count == 0
        ? 1
        : _pets.Max(pet => pet.Id) + 1;

    var caracteristicas = viewModel.Caracteristicas
        .Split(',', StringSplitOptions.TrimEntries |
StringSplitOptions.RemoveEmptyEntries);

    if (caracteristicas.Length == 0)
    {
        caracteristicas = ["Aguardando avaliação"];
    }

    var pet = new Pet(
        id,
        viewModel.Nome.Trim(),
        viewModel.Especie.Trim(),
        viewModel.Porte.Trim(),
        viewModel.IdadeAproximada,
        viewModel.Cidade.Trim(),
        viewModel.Descricao.Trim(),
        viewModel.Icone.Trim(),
        true,
        caracteristicas);

    _pets.Add(pet);

    return pet;
}
```

O filtro mostra outro ponto importante do MVC: nem todo comportamento precisa ficar dentro de cada action. Um filtro pode executar lógica antes ou depois da action, permitindo centralizar pequenas intervenções no fluxo. No projeto PetMatch.Web, o arquivo Filters/PetMatchActionFilter.cs implementa um filtro simples que adiciona informação contextual às páginas MVC.

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace PetMatch.Web.Filters;

public sealed class PetMatchActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        if (context.Controller is Controller controller)
        {
            controller.ViewData["PetMatchContexto"] = "Área MVC do PetMatch";
        }
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
    }
}

```

Esse filtro não altera dados de negócio nem substitui validações do formulário. Seu papel é demonstrar que o MVC possui pontos de extensão no fluxo das actions. Antes de a view ser renderizada, o filtro acrescenta uma informação em ViewData, que pode ser usada pelo layout ou pelas páginas. Em aplicações maiores, filtros também podem registrar eventos, aplicar políticas, tratar exceções ou padronizar comportamentos repetidos.

O controller reúne as actions da área de pets. No projeto PetMatch.Web, o arquivo Controllers/ PetsController.cs recebe o armazenamento em memória por injeção de dependência, exibe a listagem, apresenta o formulário e processa o cadastro. O ASP.NET Core cria a instância de PetsController quando uma rota MVC aponta para ele; ao encontrar o construtor PetsController(PetMemoryStore store), busca PetMemoryStore no contêiner de serviços e entrega esse objeto automaticamente ao controller. A ction POST usa [ValidateAntiForgeryToken], que ajuda a proteger formulários contra envios forjados de outro site.

A view de cadastro usa Tag Helpers para ligar os campos do formulário às propriedades do ViewModel. No projeto PetMatch.Web, o arquivo Views/Pets/Create.cshtml contém o formulário MVC e os pontos em que as mensagens de validação aparecem no navegador.

```

using Microsoft.AspNetCore.Mvc;
using PetMatch.Web.Data;
using PetMatch.Web.ViewModels;

namespace PetMatch.Web.Controllers;

public sealed class PetsController : Controller
{
    private readonly PetMemoryStore _store;

    public PetsController(PetMemoryStore store)
    {
        _store = store;
    }

    [HttpGet]
    public IActionResult Index()
    {
        var pets = _store.Listar(null, null, null);
        return View(pets);
    }

    [HttpGet]
    public IActionResult Create()
    {
        return View(new PetCreateViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(PetCreateViewModel viewModel)
    {
        if (!ModelState.IsValid)
        {
            return View(viewModel);
        }

        var pet = _store.Add(viewModel);
        TempData["MensagemSucesso"] = $"Cadastro de {pet.Nome} criado com
sucesso.";

        return RedirectToAction(nameof(Index));
    }
}

```

```

@model PetCreateViewModel

@{
    ViewData["Title"] = "Cadastrar pet";
}

<section class="mvc-panel form-panel">
    <div class="page-heading">
        <p class="eyebrow">@ViewData["PetMatchContexto"]</p>
        <h1>Cadastrar pet para adoção</h1>
        <p>Preencha os dados principais para criar um card de adoção responsável.</p>
    </div>

    <form asp-action="Create" method="post" class="pet-form">
        <div asp-validation-summary="ModelOnly" class="validation-summary">

            <label asp-for="Nome"></label>
            <input asp-for="Nome" />
            <span asp-validation-for="Nome" class="field-validation"></span>

            <label asp-for="Especie"></label>
            <select asp-for="Especie">
                <option value="">Selecione</option>
                <option value="Cachorro">Cachorro</option>
                <option value="Gato">Gato</option>
            </select>
            <span asp-validation-for="Especie" class="field-validation"></span>

            <label asp-for="Porte"></label>
            <select asp-for="Porte">
                <option value="">Selecione</option>
                <option value="Pequeno">Pequeno</option>
                <option value="Médio">Médio</option>
                <option value="Grande">Grande</option>
            </select>
            <span asp-validation-for="Porte" class="field-validation"></span>

            <label asp-for="IdadeAproximada"></label>
            <input asp-for="IdadeAproximada" type="number" min="0" max="30" />
            <span asp-validation-for="IdadeAproximada" class="field-validation">

        </div>

        <label asp-for="Cidade"></label>
        <input asp-for="Cidade" />
        <span asp-validation-for="Cidade" class="field-validation"></span>

        <label asp-for="Icone"></label>
        <input asp-for="Icone" />
        <span asp-validation-for="Icone" class="field-validation"></span>
    </form>
</div>

```

```

<label asp-for="Descricao"></label>
<textarea asp-for="Descricao" rows="4"></textarea>
<span asp-validation-for="Descricao" class="field-validation"></span>

<label asp-for="Caracteristicas"></label>
<input asp-for="Caracteristicas" placeholder="Vacinado, dócil, sociável"
/>
<span asp-validation-for="Caracteristicas" class="field-validation">
</span>

<div class="form-actions">
    <a asp-controller="Pets" asp-action="Index" class="secondary-action"
>Voltar</a>
    <button type="submit">Salvar cadastro</button>
</div>
</form>
</section>

```

O formulário demonstra o vínculo entre ViewModel, model binding e validação. Os atributos asp-for fazem os campos corresponderem às propriedades do modelo. Os elementos asp-validation-for exibem as mensagens associadas às Data Annotations. Quando o usuário envia um formulário vazio ou inválido, a action POST devolve a mesma view e o navegador mostra as mensagens próximas aos campos.

As demais views completam a experiência visual. Views/Shared/_Layout.cshtml define a estrutura comum da área MVC, carrega wwwroot/css/petmatch-mvc.css e oferece navegação para listagem e cadastro. Views/Pets/Index.cshtml renderiza os pets em cards e mostra uma mensagem de sucesso quando um cadastro válido é concluído. _ViewImports.cshtml habilita os Tag Helpers e importa os namespaces usados pelas views.

Em /Pets, o leitor deve ver cards de pets organizados em uma página estilizada, com navegação superior e acesso ao cadastro (figura 8). Em /Pets/Create, deve aparecer um formulário agradável, com campos rotulados, botão de envio e mensagens de validação visíveis quando os dados não atendem às regras declaradas (figura 9). Ao preencher dados válidos, a aplicação retorna à listagem com mensagem de sucesso e o novo pet aparece entre os cards (figura 10). Os endpoints Minimal API continuam disponíveis em caminhos como /api/pets (figura 11), /api/pets/6 (figura 12) e /api/boas-vindas/Ana (figura 13).

2.2 Views, layouts/partials e Tag Helpers

Uma aplicação MVC não produz a resposta visual apenas por meio do controller, o qual recebe a requisição, executa a action apropriada e prepara os dados, mas a apresentação final é construída pelas views. Em ASP.NET Core, uma Razor View combina HTML com expressões C# controladas, permitindo que a página receba um modelo tipado, leia informações enviadas pelo controller e gere uma resposta HTML coerente com o estado da aplicação.

No PetMatch, a área MVC já possui a listagem /Pets e o formulário /Pets/Create. O objetivo agora é tornar essa camada de apresentação mais organizada e reutilizável. A aplicação continuará com o controller `PetsController`, o `PetCreateViewModel`, as validações do formulário e os endpoints Minimal API já publicados. A evolução ocorre nas views, o layout compartilhado passa a concentrar a moldura comum das páginas, os partials reduzem repetição na interface e os Tag Helpers tornam links, formulários, validações e fragmentos reutilizáveis mais integrados ao MVC.

No projeto `PetMatch.Web`, o arquivo `Views/Shared/_Layout.cshtml` foi ajustado para servir como moldura da área MVC. O trecho a seguir mostra o uso de título dinâmico, referência ao CSS com controle de versão por Tag Helper, navegação com `asp-controller` e `asp-action`, e a chamada `@RenderBody()`, que indica o ponto em que cada view será inserida no layout.

```

<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>@ViewData["Title"] - PetMatch</title>
    <link rel="stylesheet" href="~/css/petmatch-mvc.css"
asp-append-version="true" />
</head>
<body>
    <header class="mvc-topbar">
        <a class="mvc-brand" asp-controller="Pets" asp-action="Index">PetMatch
</a>

        <nav class="mvc-nav">
            <a asp-controller="Pets" asp-action="Index">Pets disponíveis</a>
            <a asp-controller="Pets" asp-action="Create">Cadastrar pet</a>
        </nav>
    </header>

    <main class="mvc-shell">
        @RenderBody()
    </main>

    <footer class="mvc-footer">
        PetMatch - adoção responsável com informação clara.
    </footer>
</body>
</html>

```

Quadro 3

Escrito na view Razor	Gerado para o navegador
<a asp-controller="Pets" asp-action="Index">Pets disponíveis	Pets disponíveis
<a asp-controller="Pets" asp-action="Create">Cadastrar pet	Cadastrar pet
<link rel="stylesheet" href="~/css/petmatch-mvc.css" asp-append-version="true" />	Link para o CSS com versão anexada ao endereço

Os partials são fragmentos reutilizáveis de interface. Eles não substituem controllers nem actions; são peças de apresentação usadas por views para renderizar blocos menores. No PetMatch, dois partials foram criados em Views/Shared: `_MvcHero.cshtml`, para a faixa visual de apresentação das páginas MVC, e `_PetCard.cshtml`, para renderizar cada pet em formato de card. Essa escolha evita que a listagem repita o mesmo HTML sempre que precisar mostrar um pet. O partial `_PetCard.cshtml`, no projeto `PetMatch.Web`, é tipado para o modelo `Pet`. Ele recebe um `pet` e produz o card visual correspondente, com ícone, nome, descrição, metadados e características.

```

@model Pet

<article class="pet-card">
    <div class="pet-card-icon">@Model.Icone</div>

    <div class="pet-card-body">
        <h2>@Model.Nome</h2>
        <p>@Model.Descricao</p>

        <div class="pet-meta">
            <span>@Model.Especie</span>
            <span>@Model.Porte</span>
            <span>@Model.IdadeAproximada ano(s)</span>
            <span>@Model.Cidade</span>
        </div>

        <div class="pet-tags">
            @foreach (var caracteristica in Model.Caracteristicas)
            {
                <span>@caracteristica</span>
            }
        </div>
    </div>
</article>

```

A view de listagem, em Views/Pets/Index.cshtml, passa a usar `partials` e `Tag Helpers` para compor a página. Ela exibe o hero reutilizável, mostra a mensagem de sucesso quando um cadastro válido é realizado, oferece um link para o formulário de cadastro e renderiza os cards por meio do `partial` tipado.

```

@model IReadOnlyList<Pet>

@{
    ViewData["Title"] = "Pets disponíveis";
}

<partial name="_MvcHero" />

@if (TempData["MensagemSucesso"] is string mensagem)
{
    <div class="success-message">@mensagem</div>
}

<section class="page-actions">
    <a class="primary-action" asp-controller="Pets" asp-action="Create">
        Cadastrar novo pet
    </a>
</section>

<section class="pet-grid">
    @foreach (var pet in Model)
    {
        <partial name="_PetCard" model="pet" />
    }
</section>

```

Unidade II

3 PERSISTÊNCIA E CONEXÃO COM BD COM EF CORE

O banco de dados cumpre uma função distinta da memória da aplicação. A memória é rápida e adequada ao processamento imediato, mas não foi projetada para manter o histórico estável de um sistema. O banco de dados, por sua vez, oferece mecanismos de armazenamento, consulta, relacionamento, integridade e controle de transações. Ele permite que a aplicação represente dados de forma estruturada e confiável, evitando que informações essenciais sejam perdidas ou se tornem contraditórias.

Ao aproximar uma aplicação orientada a objetos de um banco de dados relacional, surge uma diferença importante de modelos. No código C#, o desenvolvedor trabalha com classes, propriedades, objetos e coleções. No banco de dados relacional, as informações são organizadas em tabelas, colunas, linhas, chaves e relacionamentos. Para fazer essa mediação se utiliza mapeamento objeto-relacional, frequentemente associado à sigla ORM (Object-Relational Mapping), e tem como objetivo permitir que entidades do domínio da aplicação sejam armazenadas e recuperadas em estruturas relacionais sem que o desenvolvedor precise escrever manualmente todas as operações de conversão.

O Entity Framework Core (EF Core) ocupa papel central nesse cenário dentro do ecossistema .NET. Ele permite descrever entidades, relacionamentos e regras de mapeamento de modo integrado ao código da aplicação, tornando possível consultar e modificar dados por meio de objetos C#. Em vez de tratar cada operação como uma sequência explícita de comandos SQL (Structured Query Language), o desenvolvedor trabalha com um modelo conceitual mais próximo da linguagem da aplicação.

Além do Entity Framework Core, o ecossistema .NET também oferece formas de acesso a dados mais próximas do SQL, como ADO.NET e Dapper. O ADO.NET representa uma camada fundamental de comunicação com bancos relacionais, trabalhando diretamente com conexões, comandos, parâmetros, leitores de dados e transações. O Dapper, por sua vez, é frequentemente descrito como um micro-ORM,

pois mantém o SQL escrito de forma explícita, mas facilita a conversão dos resultados para objetos C#.

3.1 Modelagem, DbContext, migrations, seed e pooling de conexões

Antes, o PetMatch usava PetMemoryStore, uma lista em memória criada a cada execução. Agora, passa a usar PetMatchDbContext, que representa a conexão entre a aplicação e o banco SQLite. Antes, cadastrar um pet durante a execução produzia um resultado temporário. Agora, o cadastro é gravado em petmatch.db e pode continuar disponível depois que a aplicação for encerrada e iniciada novamente.

No PetMatch, o pet deixa de ser apenas um item dentro de uma lista temporária em memória e passa a ser um registro armazenado em um banco SQLite. Essa entidade não deve ser confundida com o PetCreateViewModel. O ViewModel representa os dados recebidos pelo formulário de cadastro, ou seja, aquilo que vem da tela no momento em que o usuário preenche as informações. A entidade Pet, por sua vez, representa o dado que será salvo e consultado pela aplicação.

O DbContext é o ponto central de integração entre a aplicação e o EF Core. Ele pode ser entendido como o componente responsável por conversar com o banco de dados durante uma operação.

Para introduzir o EF Core, o projeto PetMatch.Web recebeu os pacotes Microsoft.EntityFrameworkCore.Sqlite 10.0.9, Microsoft.EntityFrameworkCore.Design 10.0.9 e SQLitePCLRaw.lib.e_sqlite3 3.50.3. A escolha por SQLite mantém o exemplo local e simples, sem exigir a instalação ou configuração de um servidor de banco de dados separado.

O arquivo appsettings.json, no projeto PetMatch.Web, recebeu uma connection string para a base do PetMatch.

```
{
  "ConnectionStrings": {
    "PetMatchConnection": "Data Source=petmatch.db;Pooling=True"
  }
}
```

Nessa configuração, Data Source=petmatch.db indica que o banco será armazenado em um arquivo chamado petmatch.db. Já a opção Pooling=True está relacionada ao reaproveitamento de conexões com o banco. O pooling de conexões permite que

conexões já criadas sejam reaproveitadas quando possível, em vez de criar uma conexão completa para cada operação.

O projeto usa `AddDbContextPool` para registrar o `PetMatchDbContext`. Esse recurso também envolve reaproveitamento, mas em outro nível. Enquanto o pooling de conexões trata do reaproveitamento das conexões usadas pelo provedor SQLite, o pooling de contexto trata do reaproveitamento controlado de instâncias do `DbContext` pelo ASP.NET Core. Em outras palavras, o primeiro está ligado ao canal de comunicação com o banco; o segundo está ligado ao objeto do EF Core que organiza o trabalho da aplicação com os dados.

```
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Data;
using PetMatch.Web.Endpoints;
using PetMatch.Web.Filters;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContextPool<PetMatchDbContext>(options =>
{
    var connectionString =
builder.Configuration.GetConnectionString("PetMatchConnection");
    options.UseSqlite(connectionString);
});

builder.Services.AddScoped<PetMatchActionFilter>();
```

Unidade II

```
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.AddService<PetMatchActionFilter>();
});

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<PetMatchDbContext>();
    dbContext.Database.Migrate();

    var seed = scope.ServiceProvider.GetRequiredService<PetMatchSeed>();
    seed.Seed();
}

app.UseHttpsRedirection();
app.UseDefaultFiles();
app.UseStaticFiles();

app.MapPetEndpoints();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller}/{action=Index}/{id?}");

app.Run();
```

O armazenamento em memória deixa de ser a principal forma de guardar os dados e o `PetMatchDbContext` passa a ser configurado para usar SQLite. A chamada `dbContext.Database.Migrate()` verifica se o banco local já está de acordo com a estrutura esperada pela aplicação. Caso existam mudanças registradas pelo EF Core, elas são aplicadas ao banco. Essas mudanças são chamadas de migrations. Uma migration funciona como um histórico controlado das alterações feitas no modelo de dados, como a criação de uma tabela, a inclusão de uma coluna ou a alteração de uma configuração. Assim, em vez de modificar o banco manualmente, a aplicação consegue atualizar sua estrutura de forma organizada.

O código obtém uma instância de `PetMatchSeed` e chama o método `Seed()`. O termo seed significa a inserção de dados iniciais. Esses dados servem para que a aplicação comece com informações mínimas, úteis para testes, demonstrações ou funcionamento básico. No contexto do `PetMatch`, o seed pode garantir que existam pets previamente cadastrados quando a aplicação for executada pela primeira vez, evitando que o usuário encontre uma base completamente vazia.

O arquivo `Data/PetMatchDbContext.cs`, no projeto `PetMatch.Web`, define o contexto do EF Core. Ele expõe `Pets` como `DbSet` e configura a entidade persistente.

```
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

public sealed class PetMatchDbContext : DbContext
{
    public PetMatchDbContext(DbContextOptions<PetMatchDbContext> options)
        : base(options)
    {
    }

    public DbSet<Pet> Pets => Set<Pet>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Pet>(entity =>
        {
            entity.HasKey(pet => pet.Id);

            entity.Property(pet => pet.Nome).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.Especie).HasMaxLength(40).IsRequired();
            entity.Property(pet => pet.Porte).HasMaxLength(30).IsRequired();
            entity.Property(pet => pet.Cidade).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.Descricao).HasMaxLength(300).IsRequired();
            entity.Property(pet => pet.Icone).HasMaxLength(12).IsRequired();
            entity.Property(pet => pet.CaracteristicasTexto).HasMaxLength(300)
                .IsRequired();
        });
    }
}
```

A entidade persistente Pet, no arquivo Models/Pet.cs do projeto PetMatch.Web, foi ajustada para funcionar com o EF Core e com as views já existentes. Ela mantém os dados usados pelos cards e preserva uma forma conveniente de expor as características como coleção para a interface.

Essa alteração também explica por que o Pet deixa de ser um record simples e passa a ser uma class com propriedades get; set;. Nos tópicos anteriores, o pet era principalmente um objeto de transporte de dados em memória. Com EF Core, ele passa a ser uma entidade persistente: o framework precisa criar instâncias ao ler linhas do banco, preencher propriedades, acompanhar alterações e decidir o que deve ser gravado em SaveChangesAsync().

```
using System.ComponentModel.DataAnnotations.Schema;

namespace PetMatch.Web.Models;

public sealed class Pet
{
    public int Id { get; set; }

    public string Nome { get; set; } = string.Empty;

    public string Especie { get; set; } = string.Empty;

    public string Porte { get; set; } = string.Empty;

    public int IdadeAproximada { get; set; }

    public string Cidade { get; set; } = string.Empty;

    public string Descricao { get; set; } = string.Empty;

    public string Icone { get; set; } = "🐾";

    public bool Disponivel { get; set; } = true;

    public string CaracteristicasTexto { get; set; } = string.Empty;

    [NotMapped]
    public string[] Caracteristicas
    {
        get => CaracteristicasTexto
            .Split('|', StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries);

        set => CaracteristicasTexto = string.Join('|', value);
    }
}
```

As migrations registram a evolução do esquema. No projeto PetMatch.Web, foi criada a pasta Migrations, com os arquivos

202607030001_CreatePetMatchSchema.cs

e

PetMatchDbContextModelSnapshot.cs. A migration inicial descreve a criação da tabela de pets a partir do modelo configurado. O snapshot guarda a forma atual do modelo para que o EF Core consiga comparar mudanças de estrutura quando novas alterações forem introduzidas.

O seed garante que a aplicação não abra vazia. No arquivo Data/PetMatchSeed.cs, o projeto cria pets iniciais quando a tabela ainda não possui registros.

```
using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

public sealed class PetMatchSeed
{
    private readonly PetMatchDbContext _dbContext;

    public PetMatchSeed(PetMatchDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public void Seed()
    {
        if (_dbContext.Pets.Any())
        {
            return;
        }

        _dbContext.Pets.AddRange(
            new Pet
            {
                Nome = "Luna",
                Espécie = "Cachorro",
                Porte = "Médio",
                IdadeAproximada = 3,
                Cidade = "Curitiba",
                Descrição = "Dócil, brincalhona e acostumada com crianças.",
                Ícone = "🐶",
                Disponível = true,
                Características = ["Vacinada", "Castrada", "Sociável"]
            },
            new Pet
            {
                Nome = "Mingau",
                Espécie = "Gato",
                Porte = "Pequeno",
                IdadeAproximada = 2,
                Cidade = "São Paulo",
                Descrição = "Calmo, curioso e ideal para apartamento.",
                Ícone = "🐱",
                Disponível = true,
                Características = ["Vermifugado", "Independente", "Carinhoso"]
            }
        );

        _dbContext.SaveChanges();
    }
}
```

O controller MVC passa a usar o PetMatchDbContext para consultar e gravar dados.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Data;
using PetMatch.Web.Models;
using PetMatch.Web.ViewModels;

namespace PetMatch.Web.Controllers;

public sealed class PetsController : Controller
{
    private readonly PetMatchDbContext _dbContext;

    public PetsController(PetMatchDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    [HttpGet]
    public async Task<IActionResult> Index()
    {
        var pets = await _dbContext.Pets
            .Where(pet => pet.Disponivel)
            .OrderBy(pet => pet.Nome)
            .ToListAsync();
        return View(pets);
    }

    [HttpGet]
    public IActionResult Create()
    {
        return View(new PetCreateViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(PetCreateViewModel viewModel)
    {
        if (!ModelState.IsValid)
        {
            return View(viewModel);
        }
    }
}
```

```

var pet = new Pet
{
    Nome = viewModel.Nome.Trim(),
    Espécie = viewModel.Espécie.Trim(),
    Porte = viewModel.Porte.Trim(),
    IdadeAproximada = viewModel.IdadeAproximada,
    Cidade = viewModel.Cidade.Trim(),
    Descrição = viewModel.Descrição.Trim(),
    Ícone = viewModel.Ícone.Trim(),
    Disponível = true,
    Características = viewModel.Características
        .Split(',',
StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries)
        );

    _dbContext.Pets.Add(pet);
    await _dbContext.SaveChangesAsync();

    TempData["MensagemSucesso"] = $"Cadastro de {pet.Nome} criado com
sucesso.";

    return RedirectToAction(nameof(Index));
}
}

```

Os endpoints Minimal API também passam a ler a mesma base persistente. No arquivo Endpoints/PetEndpoints.cs, os caminhos públicos foram preservados, mas as consultas usam PetMatchDbContext (figura 14). Dessa forma, a listagem MVC, a home e /api/pets passam a refletir os mesmos pets cadastrados.

```
api.MapGet("/pets", async (
    PetMatchDbContext dbContext,
    string? especie,
    string? porte,
    string? cidade) =>
{
    var consulta = dbContext.Pets
        .Where(pet => pet.Disponivel)
        .AsQueryable();

    if (!string.IsNullOrWhiteSpace(especie))
    {
        consulta = consulta.Where(pet => pet.Especie == especie);
    }
}
```

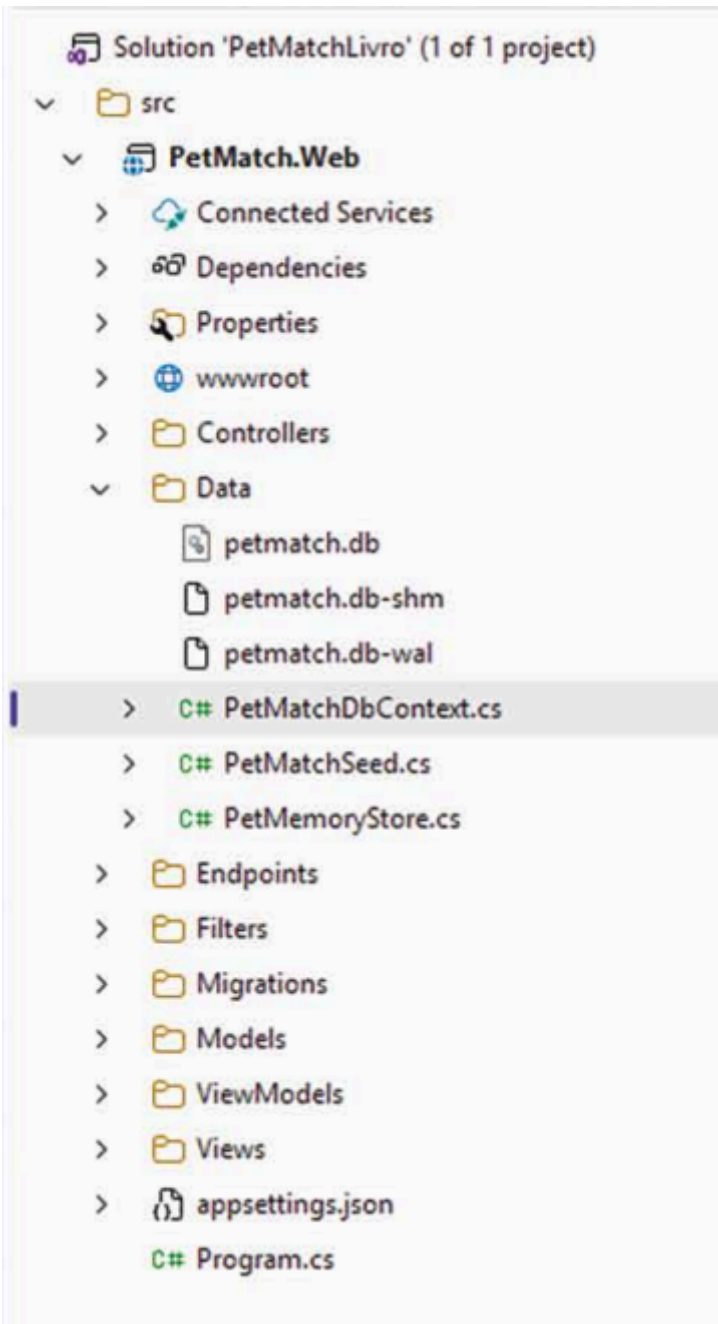
Unidade II

```
    if (!string.IsNullOrWhiteSpace(porte))
    {
        consulta = consulta.Where(pet => pet.Porte == porte);
    }

    if (!string.IsNullOrWhiteSpace(cidade))
    {
        consulta = consulta.Where(pet => pet.Cidade.Contains(cidade));
    }

    var pets = await consulta
        .OrderBy(pet => pet.Nome)
        .ToListAsync();

    return Results.Ok(pets);
});
```



```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using PetMatch.Web.Models;

namespace PetMatch.Web.Data;

12 references
public sealed class PetMatchDbContext : DbContext
{
    0 references
    public PetMatchDbContext(DbContextOptions<PetMatchDbContext> options)
        : base(options)
    {
    }

    7 references
    public DbSet<Pet> Pets => Set<Pet>();

    0 references
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var caracteristicasComparer = new ValueComparer<string[]>(
            (left, right) => left != null && right != null && left.SequenceEqual(right),
            value => value.Aggregate(0, (hash, item) => GetHashCode.Combine(hash, item.GetHashCode()),
            value => value.ToArray());

        modelBuilder.Entity<Pet>(entity =>
        {
            entity.ToTable("Pets");

            entity.HasKey(pet => pet.Id);

            entity.Property(pet => pet.Nome)
                .HasMaxLength(80)
                .IsRequired();

            entity.Property(pet => pet.Especie)
                .HasMaxLength(30)
                .IsRequired();
        });
    }
}

```

3.2 Transações/concorrência (RowVersion, DbTransaction), leituras e comandos eficientes

A persistência introduz um problema que não aparece quando os dados existem apenas em memória durante a execução: a aplicação passa a lidar com alterações duráveis, compartilhadas e potencialmente concorrentes. Em um sistema como o PetMatch, dois atendentes podem abrir o cadastro do mesmo pet, revisar informações e tentar salvar mudanças em momentos próximos. Se a aplicação simplesmente aceitar a última gravação, uma alteração pode sobrescrever outra sem que o usuário perceba. Por isso, persistir dados com EF Core exige mais do

que criar tabelas e chamar `SaveChangesAsync`; exige pensar em transações, concorrência e eficiência.

A concorrência otimista parte da ideia de que conflitos podem acontecer, mas não são frequentes o suficiente para justificar bloqueios longos. Em vez de bloquear o pet enquanto uma pessoa edita o formulário, a aplicação grava um token de versão no momento em que a tela é carregada. Ao salvar, o EF Core compara o token recebido do formulário com o token atual do banco. Se outro fluxo alterou o mesmo pet nesse intervalo, os tokens deixam de coincidir e o EF Core lança uma exceção de concorrência.

A situação pode ser imaginada em três momentos. Primeiro, o usuário A abre `/Pets/Edit/4` e recebe os dados atuais do pet, junto com uma versão do registro. Depois, o usuário B altera o mesmo pet e salva antes do usuário A. Nesse momento, a versão do banco muda. Por fim, o usuário A tenta salvar o formulário antigo. Como ele está enviando uma versão que não é mais a atual, o EF Core identifica o conflito e a aplicação mostra uma mensagem pedindo revisão dos dados.

Em bancos como SQL Server, `rowversion` pode ser uma coluna gerenciada automaticamente pelo próprio banco. O PetMatch usa SQLite local, que não possui esse recurso nativo com a mesma semântica. Por isso, neste projeto, `RowVersion` é um token de concorrência gerenciado pela aplicação, ele é armazenado como `byte[]`, configurado no EF Core como `concurrency token` e renovado pelo código antes de salvar uma atualização.

No projeto PetMatch.Web, o arquivo `Models/Pet.cs` foi alterado para incluir o token de concorrência.

```
public byte[] RowVersion { get; set; } = Guid.NewGuid().ToByteArray();
```

Essa propriedade passa a acompanhar cada pet persistido. Quando um formulário de edição é aberto, o valor atual de `RowVersion` é enviado para a view em formato transportável. Quando o usuário salva, esse valor volta para o controller, permitindo que o EF Core compare a versão originalmente lida com a versão existente no banco no momento da gravação.

No arquivo `Data/PetMatchDbContext.cs`, também no projeto PetMatch.Web, a propriedade é configurada como token de concorrência.

```
entity.Property(pet => pet.RowVersion)
    .IsConcurrencyToken()
    .IsRequired();
```

Esse mapeamento informa ao EF Core que RowVersion deve participar da verificação de concorrência. Em uma atualização, o valor original do token é incluído na condição usada para localizar o registro a ser alterado. Se nenhum registro corresponder ao identificador e à versão original, o EF Core entende que houve conflito e sinaliza o problema por meio de DbUpdateConcurrencyException.

A mudança de esquema foi registrada pela migration Migrations/202607030002_AddPetConcurrencyToken.cs, no projeto PetMatch.Web.

```
migrationBuilder.AddColumn<byte[]>(  
    name: "RowVersion",  
    table: "Pets",  
    type: "BLOB",  
    nullable: false,  
    defaultValue: Array.Empty<byte>());
```

Para transportar o token entre a view e o controller, criamos no projeto o ViewModels/ PetEditViewModel.cs. O ViewModel mantém os campos editáveis e inclui RowVersionBase64, pois um byte[] não deve ser colocado diretamente em um formulário HTML.

```
using System.ComponentModel.DataAnnotations;  
  
namespace PetMatch.Web.ViewModels;  
  
public sealed class PetEditViewModel  
{  
    public int Id { get; set; }  
  
    [Required(ErrorMessage = "Informe o nome do pet.")]  
    [StringLength(80, ErrorMessage = "O nome deve ter no máximo 80 caracteres.")]  
    public string Nome { get; set; } = string.Empty;  
  
    [Required(ErrorMessage = "Informe a cidade.")]  
    [StringLength(80, ErrorMessage = "A cidade deve ter no máximo 80 caracteres.")]  
    public string Cidade { get; set; } = string.Empty;  
  
    [Required(ErrorMessage = "Informe a descrição.")]  
    [StringLength(300, ErrorMessage = "A descrição deve ter no máximo  
300 caracteres.")]  
    public string Descricao { get; set; } = string.Empty;  
  
    public bool Disponivel { get; set; } = true;  
  
    public string RowVersionBase64 { get; set; } = string.Empty;  
}
```

O formulário não precisa expor todos os detalhes internos de Pet, mas precisa transportar o token de concorrência. A conversão para Base64 permite que o valor viaje em um campo oculto sem perder sua forma binária original.

No arquivo `Controllers/PetsController.cs`, a action GET de edição carrega o pet com `AsNoTracking`. Esse método é apropriado para leituras em que a entidade será usada apenas para montar a tela, sem necessidade de rastreamento imediato pelo contexto.

```
[HttpGet]
public async Task<IActionResult> Edit(int id)
{
    var pet = await _dbContext.Pets
        .AsNoTracking()
        .FirstOrDefaultAsync(pet => pet.Id == id);

    if (pet is null)
    {
        return NotFound();
    }

    var viewModel = new PetEditViewModel
    {
        Id = pet.Id,
        Nome = pet.Nome,
        Cidade = pet.Cidade,
        Descricao = pet.Descricao,
```

Unidade II

```
        Disponivel = pet.Disponivel,
        RowVersionBase64 = Convert.ToBase64String(pet.RowVersion)
    };

    return View(viewModel);
}
```

O controller busca apenas o pet necessário, não mantém a entidade rastreada sem necessidade e entrega à view um modelo próprio para edição.

A gravação da edição combina validação, transação explícita, renovação do token e tratamento de concorrência. No mesmo arquivo `Controllers/PetsController.cs`, a action POST trabalha com o valor original de `RowVersion`, controla a transação pelo Entity Framework Core e trata `DbUpdateConcurrencyException` com uma mensagem amigável.

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, PetEditViewModel viewModel)
{
    if (id != viewModel.Id)
    {
        return NotFound();
    }

    if (!ModelState.IsValid)
    {
        return View(viewModel);
    }

    var pet = await _dbContext.Pets.FirstOrDefaultAsync(pet => pet.Id == id);

    if (pet is null)
    {
        return NotFound();
    }

    var rowVersionOriginal = Convert.FromBase64String(viewModel.RowVersionBase64);

    _dbContext.Entry(pet)
        .Property(item => item.RowVersion)
        .OriginalValue = rowVersionOriginal;

    pet.Nome = viewModel.Nome.Trim();
    pet.Cidade = viewModel.Cidade.Trim();
    pet.Descricao = viewModel.Descricao.Trim();
    pet.Disponivel = viewModel.Disponivel;
    pet.RowVersion = Guid.NewGuid().ToByteArray();

    await using var transaction = await _dbContext.Database.
BeginTransactionAsync();

```

```

try
{
    await _dbContext.SaveChangesAsync();
    await transaction.CommitAsync();

    TempData["MensagemSucesso"] = $"Cadastro de {pet.Nome} atualizado com
sucesso.";

    return RedirectToAction(nameof(Index));
}
catch (DbUpdateConcurrencyException)
{
    await transaction.RollbackAsync();

    ModelState.AddModelError(
        string.Empty,
        "Este cadastro foi alterado por outra operação. Revise os dados
atuais antes de salvar novamente.");

    var petAtual = await _dbContext.Pets
        .AsNoTracking()
        .FirstOrDefaultAsync(item => item.Id == id);

    if (petAtual is not null)
    {
        viewModel.RowVersionBase64 =
Convert.ToBase64String(petAtual.RowVersion);
    }

    return View(viewModel);
}
}

```

O valor original de RowVersion é informado ao EF Core para que ele possa comparar a versão lida com a versão atual do banco. Antes da gravação, a aplicação renova o token. A transação criada por BeginTransactionAsync representa a unidade de trabalho; em caso de sucesso, ela é confirmada, e em caso de conflito, é desfeita.

Para que o conflito possa ser reproduzido pela própria interface, o controller também recebeu a action SimulateConcurrency.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> SimulateConcurrency(int id)
{
    var pet = await _dbContext.Pets.FirstOrDefaultAsync(pet => pet.Id == id);
```

Unidade II

```
    if (pet is null)
    {
        return NotFound();
    }
    pet.RowVersion = Guid.NewGuid().ToByteArray();

    await _dbContext.SaveChangesAsync();

    TempData["MensagemSucesso"] = "A versão do cadastro foi renovada para
simular uma atualização concorrente.";

    return RedirectToAction(nameof(Edit), new { id });
}
```

Essa action não representa uma função de negócio final do PetMatch e não deveria permanecer em uma aplicação publicada para usuários reais. Ela existe apenas para tornar visível o comportamento de concorrência durante o estudo. O leitor consegue abrir uma edição, acionar a simulação em outra aba e tentar salvar a aba original. O resultado esperado é a mensagem de conflito na view, sem perda silenciosa de dados.

```

@model PetEditViewModel

@{
    ViewData["Title"] = "Editar cadastro";
}

<partial name="_MvcHero" />

<section class="form-card">
    <form asp-action="Edit" asp-route-id="@Model.Id" method="post"
class="pet-form">
        <input asp-for="Id" type="hidden" />
        <input asp-for="RowVersionBase64" type="hidden" />

        <div asp-validation-summary="All" class="validation-summary"></div>

        <label asp-for="Nome"></label>
        <input asp-for="Nome" />
        <span asp-validation-for="Nome" class="field-validation"></span>

        <label asp-for="Cidade"></label>
        <input asp-for="Cidade" />
        <span asp-validation-for="Cidade" class="field-validation"></span>

        <label asp-for="Descricao"></label>
        <textarea asp-for="Descricao" rows="4"></textarea>
        <span asp-validation-for="Descricao" class="field-validation"></span>
    </form>
</section>

```

4

DESENVOLVIMENTO WEB COM .NET

```

<label class="check-row">
    <input asp-for="Disponivel" />
    Disponível para adoção
</label>

<div class="form-actions">
    <a asp-controller="Pets" asp-action="Index" class="secondary-action">
    Voltar</a>
    <button type="submit">Salvar alterações</button>
    </div>
</form>

<form asp-action="SimulateConcurrency" asp-route-id="@Model.Id"
method="post" class="simulation-box">
    <p>Use esta ação para simular outra operação alterando o mesmo
cadastro.</p>
    <button type="submit">Simular atualização concorrente</button>
</form>
</section>

```

A listagem também foi ajustada para conduzir o usuário ao fluxo de edição. Em Views/Shared/_PetCard.cshtml, cada card passou a exibir o link Editar cadastro, usando Tag Helpers para gerar a rota correta.

```

<a class="secondary-action"
asp-controller="Pets"
asp-action="Edit"
asp-route-id="@Model.Id">
    Editar cadastro
</a>

```

A eficiência das leituras também foi aprimorada nos endpoints. No arquivo Endpoints/ PetEndpoints.cs, a API continua nos mesmos caminhos públicos, mas passa a consultar com AsNoTracking, aplicar filtros antes da materialização e projetar PetResponse, sem expor RowVersion.

```
var consulta = dbContext.Pets
    .AsNoTracking()
    .Where(pet => pet.Disponivel);
```

Unidade II

```
if (!string.IsNullOrEmpty(especie))
{
    consulta = consulta.Where(pet => pet.Especie == especie);
}
var pets = await consulta
    .OrderBy(pet => pet.Nome)
    .Select(pet => new PetResponse(
        pet.Id,
        pet.Nome,
        pet.Especie,
        pet.Porte,
        pet.IdadeAproximada,
        pet.Cidade,
        pet.Descricao,
        pet.Icone,
        pet.Disponivel,
        pet.Caracteristicas))
    .ToListAsync();

return Results.Ok(pets);
```

O AsNoTracking evita rastreamento desnecessário em consultas somente de leitura. Os filtros são aplicados antes de ToListAsync, permitindo que o banco participe da seleção dos dados.

4 APIS REST E OPENAPI

Um mesmo sistema pode atender usuários por páginas MVC e Razor, ao mesmo tempo em que fornece dados para um aplicativo móvel ou um serviço parceiro.

O REST propõe uma forma de pensar sistemas distribuídos em torno de recursos identificáveis e representações transferidas entre cliente e servidor. Em termos práticos, isso significa que a API não deve ser entendida apenas como um conjunto de métodos remotos chamados pela rede. Ela organiza o domínio em recursos acessíveis por endereços, utiliza verbos HTTP para indicar a intenção da operação e devolve respostas com códigos de status, cabeçalhos e corpos de mensagem adequados. Essa estrutura aproveita a semântica do próprio protocolo HTTP.

Em PetMatch, uma API REST pode representar uma forma de abrir o sistema para diferentes experiências de uso. Informações sobre pets disponíveis para adoção, detalhes de um animal, registros de interesse, alterações de disponibilidade ou consultas filtradas por espécie, porte e localização poderiam ser expostas como recursos consumíveis por outros clientes. Um aplicativo móvel voltado a adotantes, um painel de acompanhamento usado por uma organização protetora ou uma integração com uma plataforma municipal de bem-estar animal poderiam depender de uma API para acessar informações do PetMatch sem reproduzir diretamente suas páginas visuais.

A noção de contrato é central para compreender APIs REST. Quando uma aplicação disponibiliza um endpoint, ela estabelece uma promessa técnica para quem o consome. Essa promessa envolve o endereço do recurso, o método HTTP esperado, os parâmetros aceitos, o formato do corpo da requisição, as validações aplicadas, os códigos de resposta possíveis e a estrutura dos dados retornados.

Os métodos HTTP desempenham papel fundamental nessa organização. Uma consulta deve ser distinguida de uma criação, uma atualização ou uma remoção, não apenas pelo nome do endereço, mas pela intenção semântica expressa na requisição. Essa distinção ajuda a tornar a API compreensível, previsível e compatível com mecanismos próprios da web, como cache, idempotência e tratamento padronizado de respostas

À medida que uma API cresce, sua documentação deixa de ser acessória e torna-se parte indispensável do contrato. Consumidores precisam saber como chamar os endpoints, quais parâmetros informar, que formato enviar, que respostas esperar e como interpretar falhas. Documentação escrita manualmente, quando desconectada do código, tende a se desatualizar com facilidade. A OpenAPI responde a esse problema ao oferecer uma especificação padronizada para descrever APIs HTTP de maneira estruturada, legível por pessoas e processável por ferramentas. Com ela, a API pode ser documentada, explorada e validada a partir de um contrato formal.

4.1 Recursos, paginação/ordenação/filtros e erros (Problem Details)

Uma API HTTP se torna mais previsível quando seus endereços representam recursos claros e suas respostas seguem contratos estáveis. No PetMatch, a coleção de pets disponível em `/api/pets` representa um recurso de listagem, enquanto `/api/pets/{id}` representa um pet específico. Essa organização permite que o cliente da API compreenda a intenção da chamada pelo próprio endereço e pelo método HTTP usado.

O conceito de recurso ajuda a separar a API de detalhes internos da aplicação. A entidade persistente `Pet` possui propriedades necessárias ao EF Core, inclusive dados que não devem ser publicados, como o token de concorrência `RowVersion`. Já a resposta HTTP deve expor apenas o que o cliente precisa conhecer. Por isso, o PetMatch passa a usar contratos próprios em `Contracts/PetResponse.cs`, `Contracts/PetQueryParameters.cs` e `Contracts/PagedResponse.cs`.

```
namespace PetMatch.Web.Contracts;

public sealed record PagedResponse<T>(
    IReadOnlyList<T> Items,
    int Page,
    int PageSize,
    int TotalItems,
    int TotalPages);
```

No projeto PetMatch.Web, o arquivo `Contracts/PetQueryParameters.cs` concentra os parâmetros aceitos por `/api/pets`.

```

namespace PetMatch.Web.Contracts;

public sealed class PetQueryParameters
{
    private const int MaxPageSize = 20;

    public int Page { get; set; } = 1;

    private int _pageSize = 6;

    public int PageSize
    {
        get => _pageSize;
        set => _pageSize = Math.Clamp(value, 1, MaxPageSize);
    }

    public string? Espécie { get; set; }

    public string? Porte { get; set; }

    public string? Cidade { get; set; }

    public bool? Disponível { get; set; }

    public bool? DisponívelParaAdocao { get; set; }

    public string? Sort { get; set; }
}

```

A resposta de cada pet também foi formalizada. No projeto PetMatch.Web, o arquivo Contracts/ PetResponse.cs define o DTO (Data Transfer Object) publicado pela API.

```

namespace PetMatch.Web.Contracts;

public sealed record PetResponse(
    int Id,
    string Nome,
    string Espécie,
    string Porte,
    int IdadeAproximada,
    string Cidade,
    string Descriçao,

```

Unidade II

```

    string Icone,
    bool Disponível,
    string[] Características);

```

No arquivo Endpoints/PetEndpoints.cs, o endpoint GET /api/pets passa a aplicar filtros, ordenação, paginação e projeção. A consulta usa AsNoTracking, pois a listagem é uma leitura sem alteração de estado e só materializa os resultados depois de aplicar os critérios recebidos.

```
api.MapGet("/pets", async (
    PetMatchDbContext dbContext,
    [AsParameters] PetQueryParameters query) =>
{
    var page = Math.Max(query.Page, 1);

    var consulta = dbContext.Pets
        .AsNoTracking()
        .AsQueryable();
    var disponivel = query.Disponivel ?? query.DisponivelParaAdocao;

    if (disponivel.HasValue)
    {
        consulta = consulta.Where(pet => pet.Disponivel == disponivel.Value);
    }

    if (!string.IsNullOrEmpty(query.Especie))
    {
        consulta = consulta.Where(pet => pet.Especie == query.Especie);
    }

    if (!string.IsNullOrEmpty(query.Porte))
    {
        consulta = consulta.Where(pet => pet.Porte == query.Porte);
    }

    if (!string.IsNullOrEmpty(query.Cidade))
    {
        consulta = consulta.Where(pet => pet.Cidade.Contains(query.Cidade));
    }

    consulta = query.Sort?.ToLowerInvariant() switch
    {
        "cidade" => consulta.OrderBy(pet => pet.Cidade).ThenBy(pet => pet.Nome),
        "idade" => consulta.OrderBy(pet => pet.IdadeAproximada).ThenBy(pet =>
pet.Nome),
        _ => consulta.OrderBy(pet => pet.Nome)
    };

    var totalItems = await consulta.CountAsync();
    var totalPages = (int)Math.Ceiling(totalItems / (double)query.PageSize);

    var pets = await consulta
        .Skip((page - 1) * query.PageSize)
        .Take(query.PageSize)
        .Select(pet => new PetResponse(
            pet.Id,
            pet.Nome,
            pet.Especie,
            pet.Porte,
            pet.IdadeAproximada,
            pet.Cidade,
            pet.Descricao,
            pet.Icone,
            pet.Disponivel,
            pet.Caracteristicas))
        .ToListAsync();

    return Results.Ok(new PagedResponse<PetResponse>(
        pets,
        page,
        query.PageSize,
        totalItems,
        totalPages));
});
```

Os erros também fazem parte do contrato. Uma API que retorna formatos diferentes para cada falha obriga o cliente a tratar exceções de maneira frágil. O padrão Problem Details oferece um formato reconhecível para erro HTTP, com campos como title, status e detail.

No projeto PetMatch.Web, o suporte a Problem Details é registrado no arquivo Program.cs, junto aos demais serviços da aplicação e antes da chamada a builder.Build():

```
builder.Services.AddProblemDetails();
```

Esse registro habilita os serviços relacionados à produção de respostas de erro padronizadas. Nos endpoints, o uso de Results.Problem permite definir o status HTTP e apresentar informações compreensíveis sobre a falha.

```
api.MapGet("/pets/{id:int}", async (PetMatchDbContext dbContext, int id) =>
{
    var pet = await dbContext.Pets
        .AsNoTracking()
        .Where(pet => pet.Id == id)
        .Select(pet => new PetResponse(
            pet.Id,
            pet.Nome,
            pet.Especie,
            pet.Porte,
            pet.IdadeAproximada,
            pet.Cidade,
            pet.Descricao,
            pet.Icone,
            pet.Disponivel,
            pet.Caracteristicas))
        .FirstOrDefaultAsync();
    return pet is null
        ? Results.Problem(
            title: "Pet não encontrado.",
            detail: $"Não existe pet cadastrado com o identificador {id}.",
            statusCode: StatusCodes.Status404NotFound)
        : Results.Ok(pet);
});
```

Para tornar a API explorável sem ferramentas externas, criamos no projeto a página estática wwwroot/api.html e o script wwwroot/js/petmatch-api.js. Essa página contém controles para espécie, porte, cidade, disponibilidade, ordenação, página e tamanho da página.

```
async function carregarPets() {  
  const parametros = new URLSearchParams(new FormData(formulario));  
  const resposta = await fetch(`/api/pets?${parametros.toString()}`);  
  const conteudo = await resposta.json();  
  
  if (!resposta.ok) {  
    mostrarProblemDetails(conteudo);  
    return;  
  }  
  
  renderizarMetadados(conteudo);  
  renderizarPets(conteudo.items);  
}
```

4.2 Documentação com Swagger e geração de cliente TypeScript (NSwag)

Uma API não é apenas um conjunto de rotas que respondem em JSON. Quando ela passa a ser consumida por uma interface, por outro serviço ou por uma equipe diferente daquela que a implementou, ela precisa de um contrato explícito. Esse contrato descreve quais recursos existem, quais parâmetros cada operação aceita, quais respostas podem ser produzidas, quais códigos HTTP indicam sucesso ou falha e qual é a forma dos dados trafegados.

O Swagger é frequentemente usado como sinônimo de OpenAPI, mas convém distinguir os termos. OpenAPI é a especificação do contrato; Swagger é o conjunto de ferramentas e a interface interativa que permitem visualizar e experimentar esse contrato no navegador. O Swagger entra como instrumento de desenvolvimento, restrito ao ambiente de desenvolvimento, sem botão ou link nas telas públicas da aplicação.

A geração de cliente TypeScript surge quando o contrato OpenAPI deixa de ser apenas documentação e passa a orientar código. Em uma integração manual, o front-end tende a reescrever URLs, nomes de parâmetros, tipos de resposta e formatos de erro. Essa duplicação é frágil: se a API muda e o cliente não acompanha, a divergência aparece em tempo de execução. Com uma ferramenta como NSwag, o contrato OpenAPI pode gerar um arquivo TypeScript com tipos e métodos correspondentes às operações descritas.

```
builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc(
        PetMatchOpenApi.DocumentName,
        PetMatchOpenApi.CreateInfo());
});
```

É importante distinguir dois momentos da documentação. Quando a aplicação está em execução, o Swagger expõe o contrato em `/swagger/v1/swagger.json` e apresenta a interface interativa em `/swagger`. Esse documento é produzido para consulta no navegador durante o desenvolvimento. Ele não deve ser confundido com um arquivo físico gerado durante a compilação do projeto. Essa diferença é essencial para entender onde o NSwag entra. A tela do Swagger permite visualizar e testar a API, mas não é ela que gera diretamente o cliente TypeScript. Para que o NSwag seja executado durante o build, o projeto precisa ter antes um documento OpenAPI gravado em disco. No PetMatch, esse arquivo será gerado em `obj/openapi/v1.json`. Somente depois disso o NSwag lê esse contrato e produz o arquivo TypeScript em `Clients/TypeScript/petmatch-api-client.ts`.

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();

    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint(
            "/swagger/v1/swagger.json",
            "PetMatch API v1");
    });
}
```

```

using Microsoft.OpenApi.Models;

namespace PetMatch.Web.OpenApi;

public static class PetMatchOpenApi
{
    public const string DocumentName = "v1";

    public static OpenApiInfo CreateInfo()
    {
        return new OpenApiInfo
        {
            Title = "PetMatch API",
            Version = "v1",
            Description = "Contrato HTTP do PetMatch para consulta de pets, manifestações de interesse e respostas padronizadas."
        };
    }
}

```

O NSwag entra em uma etapa posterior, não substitui a tela do Swagger nem aparece como uma página visual da aplicação. Sua função é ler o documento OpenAPI gerado pela API e produzir código TypeScript a partir dele. No PetMatch, essa configuração estará no arquivo nswag.json, que define o documento de entrada e o arquivo de saída do cliente TypeScript. Conforme detalharemos adiante, o resultado esperado é o arquivo Clients/TypeScript/petmatch-api-client.ts, contendo uma classe PetMatchApiClient com métodos e tipos derivados do contrato da API. Assim, Swagger e NSwag não fazem a mesma coisa. Swagger torna o contrato visível e testável no navegador. NSwag usa esse contrato como fonte para gerar código cliente.

Unidade III

5 INTEGRAÇÃO REACT/NODE – CONSUMO DA API

A evolução de uma aplicação web frequentemente conduz à separação entre a experiência visual oferecida ao usuário e os serviços responsáveis por processar dados, aplicar regras e expor funcionalidades. Front-end concentra a construção da interface, a interação com o usuário, a composição visual das telas e a resposta imediata aos eventos no navegador. De outro, a API concentra contratos HTTP, validações, persistência, regras de aplicação e integração com o banco de dados.

O objetivo não é tratar React, Node e ASP.NET Core como ilhas tecnológicas, mas compreender como cada elemento contribui para uma solução web moderna. O front-end organiza a interação e a apresentação; o ambiente Node sustenta o desenvolvimento e a construção da interface; a API oferece contratos, processamento e acesso aos dados.

5.1 Setup do front-end (Node LTS, Vite/Next), CORS e proxy de desenvolvimento

O objetivo principal deste tópico é compreender como uma aplicação web moderna pode ser dividida em duas partes que conversam entre si: um back-end, responsável pelas regras, dados e endpoints HTTP; e um front-end, responsável pela interface com a qual o usuário interage no navegador. O back-end, nesse cenário, não precisa montar toda a tela. Ele expõe dados e operações por meio de uma API. O front-end recebe esses dados e decide como apresentá-los ao usuário.

Neste tópico, Node.js não substitui o ASP.NET Core. O PetMatch continua usando ASP.NET Core no back-end. O papel do Node.js é apoiar o desenvolvimento do front-end: instalar pacotes, executar o Vite, compilar TypeScript e preparar os arquivos que o navegador vai carregar.

Pacote	Função
react	Biblioteca usada para construir interfaces baseadas em componentes
react-dom	Integra o React ao DOM do navegador
vite	Ferramenta de desenvolvimento e empacotamento do front-end
typescript	Permite escrever JavaScript com tipos estáticos
@vitejs/plugin-react	Integra o React ao Vite

O Vite é uma ferramenta para desenvolver e empacotar aplicações front-end. Durante o desenvolvimento, ele inicia um servidor local rápido. Esse servidor entrega os arquivos da aplicação React ao navegador e atualiza a tela quando o código muda. Sem uma ferramenta como o Vite, o fluxo de desenvolvimento seria mais trabalhoso. O desenvolvedor teria que lidar manualmente com compilação, organização de módulos, atualização do navegador e preparação dos arquivos finais.

Assim, durante o desenvolvimento, a interface React ficará disponível em `http://localhost:5173`. O back-end ASP.NET Core continuará em sua própria porta, por exemplo: `http://localhost:5213`. Essa diferença de portas é importante para entender CORS e proxy.

Para o navegador, uma origem é formada por protocolo, domínio e porta. Assim, estes dois endereços são origens diferentes: `http://localhost:5173` e `http://localhost:5213`. Embora ambos estejam na mesma máquina, as portas são diferentes. Para o navegador, isso já basta para tratar as duas URLs como origens distintas. Essa regra existe por segurança. Uma página carregada de uma origem não pode simplesmente acessar qualquer recurso de outra origem sem permissão. O mecanismo que controla essa permissão se chama CORS, que significa Cross-Origin Resource Sharing. Em português, podemos entender como compartilhamento de recursos entre origens diferentes.

Quando a interface React do PetMatch executa em `http://localhost:5173` e chama a API ASP.NET Core em `http://localhost:5213`, o navegador verifica se o back-end permite essa comunicação. Se o back-end não declarar essa permissão, a chamada pode ser bloqueada pelo navegador.

Um proxy é um intermediário. No contexto deste tópico, o proxy de desenvolvimento permite que chamadas feitas pela interface React sejam encaminhadas ao back-end ASP.NET Core. O front-end pode chamar `/api/pets` em vez de escrever a URL completa do back-end.

O React pode ser usado com diferentes ferramentas. Duas escolhas conhecidas são Vite e Next.js. O Next.js é um framework completo. Ele oferece roteamento próprio, renderização no servidor, convenções de projeto e recursos avançados para aplicações React. Ele é muito útil quando a aplicação precisa dessas características desde o início.

No projeto PetMatch.Frontend, o arquivo `PetMatch.Frontend.esproj` usa o SDK JavaScript do Visual Studio:

```
<Project Sdk="Microsoft.VisualStudio.JavaScript.Sdk">  
  <PropertyGroup>
```

Unidade III

```
    <StartupCommand>npm run dev</StartupCommand>  
    <JavaScriptTestRoot>src\</JavaScriptTestRoot>  
    <ShouldRunNpmInstall>true</ShouldRunNpmInstall>  
  </PropertyGroup>  
</Project>
```

Esse arquivo não representa uma página da aplicação, mas serve para que o Visual Studio reconheça o projeto front-end e saiba como trabalhar com ele. A propriedade `StartupCommand` indica o comando usado para iniciar o servidor de desenvolvimento. O comando `npm run dev` normalmente aciona o Vite. A propriedade `ShouldRunNpmInstall` permite que o Visual Studio restaure dependências JavaScript quando necessário.

A ideia central é esta: o arquivo `.esproj` ajuda a IDE (Integrated Development Environment ou Ambiente de Desenvolvimento Integrado) a tratar o front-end como parte da solução, mesmo que ele use ferramentas diferentes do back-end ASP.NET Core. O fluxo de execução do PetMatch usa o projeto ASP.NET Core como ponto de partida. O leitor inicia PetMatch.Web no Visual Studio, e o SPA (Single Page Application) proxy aciona o servidor Vite na pasta do front-end. No arquivo `PetMatch.Web.csproj`, a configuração do SPA proxy fica assim:

Propriedade	Significado
<code>SpaRoot</code>	Indica onde está o projeto front-end
<code>SpaProxyServerUrl</code>	Informa a URL do servidor Vite
<code>SpaProxyLaunchCommand</code>	Define o comando usado para iniciar o front-end

Com essa configuração, o leitor não precisa iniciar manualmente dois processos separados no primeiro contato com a aplicação. Ao executar o back-end, o Visual Studio consegue acionar também o fluxo do front-end. Como a interface React executa em `http://localhost:5173` e o back-end ASP.NET Core executa em outra

porta, o back-end precisa permitir essa origem durante o desenvolvimento. No arquivo Program.cs, criamos uma política CORS:

```
const string frontendDevelopmentPolicy = "PetMatchFrontendDev";

builder.Services.AddCors(options =>
{
    options.AddPolicy(frontendDevelopmentPolicy, policy =>
    {
        policy
```

124

DESENVOLVIMENTO WEB COM .NET

```
        .WithOrigins("http://localhost:5173")
        .AllowAnyHeader()
        .AllowAnyMethod();
    });
});
```

Esse trecho registra uma política chamada `PetMatchFrontendDev`. Ela permite chamadas vindas da origem do Vite. Também permite cabeçalhos e métodos HTTP necessários para as chamadas feitas pela interface. Depois de registrar a política, ela precisa ser aplicada ao pipeline da aplicação:

```
if (app.Environment.IsDevelopment())
{
    app.UseCors(frontendDevelopmentPolicy);
}
```

O arquivo `vite.config.ts`, no projeto `PetMatch.Frontend`, configura o servidor de desenvolvimento do Vite, `vite.config.ts`, no projeto `PetMatch.Frontend` :

```

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

const backendUrl = 'http://localhost:5213';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: backendUrl,
        changeOrigin: true,
        secure: false
      },
      '/Pets': {
        target: backendUrl,
        changeOrigin: true,
        secure: false
      },
      '/css': {
        target: backendUrl,
        changeOrigin: true,
        secure: false
      },
      '/js': {
        target: backendUrl,
        changeOrigin: true,
        secure: false
      }
    }
  }
});

```

A propriedade proxy define quais caminhos devem ser encaminhados para o back-end.

Caminho	Por que encaminhar
/api	Permite que a home React chame os endpoints da API
/Pets	Permite abrir páginas ASP.NET de cadastro, listagem e edição
/css	Permite carregar estilos das páginas ASP.NET quando abertas pela porta do Vite
/js	Permite carregar scripts das páginas ASP.NET quando abertas pela porta do Vite

A home React pode chamar /api/pets?page=1&pageSize=4 e o Vite encaminha a requisição ao back-end. Da mesma forma, links como /Pets/Create e /Pets/Edit/1 continuam funcionando durante o desenvolvimento.

```
export async function listarPets(query: PetQuery):  
Promise<PagedResponse<PetResponse>> {  
  const params = new URLSearchParams();  
  
  params.set('page', String(query.page));  
  params.set('pageSize', String(query.pageSize));  
  
  if (query.especie) {  
    params.set('especie', query.especie);  
  }  
  
  if (query.porte) {  
    params.set('porte', query.porte);  
  }  
  
  if (query.cidade) {  
    params.set('cidade', query.cidade);  
  }  
  
  const response = await fetch(`/api/pets?${params.toString()}`);  
  
  if (!response.ok) {  
    throw new Error('Não foi possível carregar os pets agora.');  }  
  
  return response.json();  
}
```

```
import { useEffect, useState } from 'react';
import { listarPets, PetResponse } from '../api/petmatchClient';
import { Hero } from '../components/Hero';
```

Unidade III

```
import { PetCard } from '../components/PetCard';
import { PetFilters } from '../components/PetFilters';
import { Pagination } from '../components/Pagination';

export function App() {
  const [pets, setPets] = useState<PetResponse[]>([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [especie, setEspecie] = useState('');
  const [porte, setPorte] = useState('');
  const [cidade, setCidade] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    setLoading(true);
    setError('');

    listarPets({ page, pageSize: 4, especie, porte, cidade })
      .then(result => {
        setPets(result.items);
        setTotalPages(result.totalPages);
      })
      .catch(() => setError('Não foi possível carregar os pets agora. Tente novamente em instantes.'))
      .finally(() => setLoading(false));
  }, [page, especie, porte, cidade]);

  return (
    <main className="app-shell">
      <Hero totalPets={pets.length} />

      <PetFilters
        especie={especie}
        porte={porte}
        cidade={cidade}
        onEspecieChange={setEspecie}
        onPorteChange={setPorte}
        onCidadeChange={setCidade}
        onPageReset={() => setPage(1)}
      />

      {loading && <p className="status-card">Buscando pets disponíveis...</p>}
      {error && <p className="status-card status-error">{error}</p>}

      <section className="pet-grid">
        {pets.map(pet => <PetCard key={pet.id} pet={pet} />)}
      </section>

      <Pagination page={page} totalPages={totalPages} onPageChange={setPage} />
    </main>
  );
}
```

O componente Hero, em src/components/Hero.tsx, apresenta a proposta da tela e oferece ações diretas:

```

type HeroProps = {
  totalPets: number;
};

export function Hero({ totalPets }: HeroProps) {
  return (
    <section className="hero">
      <div>
        <p className="eyebrow">Adoção responsável</p>
        <h1>Encontre um pet para fazer parte da sua rotina.</h1>
        <p>
          Conheça animais disponíveis, filtre por perfil e avance para o cadastro
          ou gerenciamento quando encontrar uma boa combinação.
        </p>

        <div className="hero-actions">
          <a href="/Pets/Create" className="primary-action">Cadastrar pet</a>
          <a href="/Pets" className="secondary-action">Gerenciar pets</a>
        </div>
      </div>
    </section>
  );
}

```

Unidade III

```

    <div className="hero-card">
      <strong>{totalPets}</strong>
      <span>pets exibidos nesta busca</span>
    </div>
  </section>
);
}

```

PetCard.tsx, recebe um pet e renderiza seus dados principais:

PetCard.tsx, recebe um pet e renderiza seus dados principais:

```
import { PetResponse } from '../api/petmatchClient';

type PetCardProps = {
  pet: PetResponse;
};

export function PetCard({ pet }: PetCardProps) {
  return (
    <article className="pet-card">
      <div className="pet-icon">{pet.icone}</div>

      <div className="pet-card-content">
        <h2>{pet.nome}</h2>
        <p>{pet.descricao}</p>

        <div className="pet-meta">
          <span>{pet.especie}</span>
          <span>{pet.porte}</span>
          <span>{pet.idadeAproximada} ano(s)</span>
          <span>{pet.cidade}</span>
        </div>

        <div className="pet-tags">
          {pet.caracteristicas.map(caracteristica => (
            <span key={caracteristica}>{caracteristica}</span>
          ))}
        </div>

        <a className="card-action" href={`/Pets/Edit/${pet.id}`}>
          Editar cadastro
        </a>
      </div>
    </article>
  );
}
```

O PetCard não precisa saber como buscar dados, nem como funciona a paginação, nem como os filtros foram escolhidos. Ele recebe um objeto pet e exibe esse objeto na tela. O link Editar cadastro abre uma página ASP.NET existente. Assim, o front-end React e as páginas tradicionais convivem na mesma aplicação durante esta etapa do livro.

5.2 Fetch/axios, estados de requisição, roteamento e páginas de lista/detalhe

No front-end, fetch é a API nativa do navegador para realizar requisições HTTP. Ela permite chamar um endpoint, aguardar a resposta, verificar o status HTTP e converter o corpo para JSON quando a resposta traz dados estruturados. Axios é uma biblioteca bastante usada no ecossistema JavaScript para o mesmo tipo de

tarefa, oferecendo conveniências como configuração centralizada, interceptadores e tratamento uniforme de respostas.

No projeto PetMatch.Frontend, o arquivo src/main.tsx envolve a aplicação com BrowserRouter. Esse componente fornece ao React Router o contexto necessário para interpretar a URL do navegador e renderizar a página correspondente.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { App } from './App';
import './styles/petmatch.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

[App.ts](#):

```
import { Route, Routes } from 'react-router-dom';
import { PetListPage } from './pages/PetListPage';
import { PetDetailsPage } from './pages/PetDetailsPage';

export function App() {
  return (
    <Routes>
      <Route path="/" element={<PetListPage />} />
      <Route path="/pets/:id" element={<PetDetailsPage />} />
    </Routes>
  );
}
```

```

import { useEffect, useState } from 'react';
import { listarPets, PetResponse } from '../api/petmatchClient';
import { EmptyState } from '../components/EmptyState';
import { Hero } from '../components/Hero';
import { Pagination } from '../components/Pagination';
import { PetCard } from '../components/PetCard';
import { PetFilters } from '../components/PetFilters';
import { RequestStatus } from '../components/RequestStatus';

export function PetListPage() {
  const [pets, setPets] = useState<PetResponse[]>([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [especie, setEspecie] = useState('');
  const [porte, setPorte] = useState('');
  const [cidade, setCidade] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    setLoading(true);
    setError('');

    listarPets({ page, pageSize: 4, especie, porte, cidade })
      .then(result => {
        setPets(result.items);
        setTotalPages(result.totalPages);
      })
      .catch(() => setError('Não foi possível carregar os pets agora.'))
      .finally(() => setLoading(false));
  }, [page, especie, porte, cidade]);

  return (
    <main className="app-shell">
      <Hero totalPets={pets.length} />

      <PetFilters
        especie={especie}
        porte={porte}
        cidade={cidade}
        onEspecieChange={setEspecie}
        onPorteChange={setPorte}
        onCidadeChange={setCidade}
        onPageReset={() => setPage(1)}
      />

      {loading && <RequestStatus title="Buscando pets" description="Estamos carregando os pets disponíveis." />}
      {error && <RequestStatus tone="error" title="Não foi possível carregar" description={error} />}

      {!loading && !error && pets.length === 0 && (
        <EmptyState title="Nenhum pet encontrado" description="Ajuste os filtros para ver outras possibilidades." />
      )}

      <section className="pet-grid">
        {pets.map(pet => <PetCard key={pet.id} pet={pet} />)}
      </section>

      <Pagination page={page} totalPages={totalPages} onPageChange={setPage} />
    </main>
  );
}

```

O card de pet também passa a participar da navegação React. No projeto PetMatch.Frontend, o arquivo `src/components/PetCard.tsx` adiciona a ação Ver detalhes, apontando para `/pets/{id}`, apontando para a rota ASP.NET de gerenciamento.

```
import { Link } from 'react-router-dom';
import { PetResponse } from '../api/petmatchClient';

type PetCardProps = {
  pet: PetResponse;
};

export function PetCard({ pet }: PetCardProps) {
  return (
    <article className="pet-card">
```

38

DESENVOLVIMENTO WEB COM .NET

```
<div className="pet-icon">{pet.icone}</div>
<div className="pet-card-content">
  <h2>{pet.nome}</h2>
  <p>{pet.descricao}</p>

  <div className="pet-meta">
    <span>{pet.especie}</span>
    <span>{pet.porte}</span>
    <span>{pet.idadeAproximada} ano(s)</span>
    <span>{pet.cidade}</span>
  </div>

  <div className="card-actions">
    <Link className="card-action" to={` /pets/${pet.id}`}>Ver detalhes</Link>
    <a className="card-action secondary" href={` /Pets/Edit/${pet.id}`}>
Editar cadastro</a>
  </div>
</div>
</article>
);
}
```

A página de detalhe fica em src/pages/PetDetailsPage.tsx:

```
import { Link, useParams } from 'react-router-dom';
import { useEffect, useState } from 'react';
import { obterPet, PetResponse } from '../api/petmatchClient';
import { RequestStatus } from '../components/RequestStatus';

export function PetDetailsPage() {
  const { id } = useParams();
  const [pet, setPet] = useState<PetResponse | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    const petId = Number(id);

    if (!Number.isInteger(petId) || petId <= 0) {
      setError('O endereço informado não corresponde a um pet válido.');
```

setLoading(false);

return;

}

obterPet(petId)

.then(setPet)

.catch(() => setError('Não encontramos este pet no momento.'))

.finally(() => setLoading(false));

}, [id]);

if (loading) {

return <RequestStatus title="Carregando detalhes" description="Estamos buscando as informações deste pet." />;

}

if (error || pet === null) {

return <RequestStatus tone="error" title="Pet não encontrado"

description={error} />;

}

return (

<main className="details-shell">

<Link to="/" className="back-link">Voltar para a lista</Link>

<section className="details-card">

<div className="details-icon">{pet.icone}</div>

<div>

<p className="eyebrow">{pet.especie} para adoção</p>

<h1>{pet.nome}</h1>

<p>{pet.descricao}</p>

```

<div className="pet-meta">
  <span>{pet.porte}</span>
  <span>{pet.idadeAproximada}</span> ano(s)</span>
  <span>{pet.cidade}</span>
</div>

<div className="pet-tags">
  {pet.caracteristicas.map(caracteristica => (
    <span key={caracteristica}>{caracteristica}</span>
  ))}
</div>

<div className="details-actions">
  <a className="primary-action" href={` /Pets/Edit/${pet.id}`}>Editar
cadastro</a>
  <a className="secondary-action" href="/Pets/Create">Cadastrar outro
pet</a>
</div>
</div>
</section>
</main>
);
}

```

6 AUTENTICAÇÃO E AUTORIZAÇÃO PONTA A PONTA

À medida que um sistema passa a receber dados pessoais, registrar ações, proteger funcionalidades administrativas e oferecer experiências específicas para diferentes perfis de usuário, torna-se necessário estabelecer mecanismos confiáveis de identidade e controle de acesso. Autenticação e autorização formam a base dessa proteção. A autenticação responde à pergunta sobre quem está utilizando a aplicação; a autorização responde à pergunta sobre o que essa pessoa pode fazer depois de identificada.

A autenticação representa o processo pelo qual a aplicação verifica a identidade de um usuário. Em uma aplicação web, isso normalmente envolve credenciais, provedores de identidade, emissão de sessão, cookies, tokens ou outros mecanismos que permitam reconhecer uma pessoa ou um sistema em requisições posteriores.

A autorização, por sua vez, depende da identidade, mas não se confunde com ela. Saber quem é o usuário não significa permitir automaticamente todas as operações. Um adotante pode consultar pets disponíveis e registrar interesse em adoção, mas

não deve necessariamente alterar cadastros institucionais. Uma organização responsável pode administrar informações sobre animais sob seus cuidados, mas talvez não tenha permissão para modificar dados de outra instituição. Um administrador pode exercer funções mais amplas, desde que essas permissões estejam claramente definidas. A autorização transforma a identidade em decisões de acesso, aplicando regras sobre recursos, ações e perfis.

O front-end precisa compreender o estado de autenticação, apresentar corretamente telas de login e saída, adaptar a navegação conforme o perfil do usuário e lidar com respostas de acesso negado ou sessão expirada. A API, ao mesmo tempo, precisa expressar suas decisões por meio de respostas HTTP adequadas, distinguindo falta de autenticação, ausência de permissão e erros de validação.

Em muitos casos, a autorização não depende apenas de um papel geral, mas da relação entre usuário e recurso. Uma organização pode alterar os dados de pets sob sua responsabilidade, mas não de todos os pets cadastrados no sistema. Um adotante pode visualizar suas próprias manifestações de interesse, mas não as de outros usuários. Um recurso, portanto, não é protegido apenas por sua categoria, mas pelo vínculo que mantém com a identidade autenticada.

A autenticação informa quem está realizando a ação; a autorização determina se a ação é permitida; e a validação verifica se os dados enviados respeitam as regras esperadas.

6.1 ASP.NET Identity, cookies × JWT e roles/claims

Identidade é a representação de uma pessoa ou entidade dentro do sistema, normalmente associada a informações como identificador, e-mail, nome e dados de perfil. Autenticação é o processo pelo qual a aplicação confirma que o usuário é quem afirma ser, em geral por meio de credenciais, login externo, certificado ou outro mecanismo reconhecido. Autorização é a decisão tomada depois da autenticação: a aplicação verifica se aquele usuário pode executar determinada ação, abrir uma página ou acessar um recurso.

O ASP.NET Identity é a infraestrutura do ASP.NET Core para lidar com usuários, senhas, roles, claims, tokens internos, cookies de autenticação e armazenamento

desses dados com Entity Framework Core. Ele fornece serviços para criar usuários, validar senhas, entrar e sair da aplicação, associar usuários a papéis e consultar informações de identidade durante a execução de uma requisição. Ao ser integrado ao EF Core, Identity cria tabelas próprias no banco para usuários, roles, claims, logins e relacionamentos.

Conceito	O que é no ASP.NET Core Identity	Para que serve	Exemplo prático
Usuário	A entidade que representa uma pessoa ou conta no sistema, geralmente baseada em IdentityUser	Guardar dados como Id, UserName, Email, telefone, confirmação de e-mail e outros campos de perfil	Um registro na tabela AspNetUsers para ana@email.com
Senha	Credencial secreta do usuário, armazenada como hash, não como texto puro	Permitir autenticação local com e-mail/usuário e senha	O usuário digita a senha; o Identity compara o hash calculado com o hash salvo
Role	Um papel, grupo ou função atribuída ao usuário, como Admin, Manager ou User	Facilitar autorização baseada em categorias amplas de acesso	[Authorize(Roles = "Admin")] restringe uma página a administradores
Claim	Uma declaração sobre o usuário: um par tipo/valor, como Department = Finance ou Permission = CanEditInvoices	Representar informações ou permissões mais granulares que roles	Um usuário pode ter a claim Permission:EditProducts
Login	O processo ou vínculo usado para autenticar o usuário. Pode ser login local ou externo	Associar uma conta Identity a credenciais locais ou provedores externos	Login com senha local, Google, Microsoft, GitHub etc.
Tokens internos	Tokens gerados pelo Identity para operações específicas, como confirmação de e-mail, redefinição de senha e autenticação de dois fatores	Executar ações sensíveis sem expor senha ou estado diretamente	Link de "redefinir senha" contém um token temporário validado pelo Identity
Cookies de autenticação	Cookie emitido após login bem-sucedido, contendo uma identidade autenticada, normalmente com claims	Manter o usuário logado entre requisições HTTP	Depois do login, o navegador envia o cookie e o ASP.NET Core reconhece o usuário

A autenticação por cookie é uma forma tradicional e eficiente de manter uma sessão web no navegador. Depois que o usuário entra com sucesso, o servidor emite um cookie protegido. Em requisições seguintes, o navegador envia esse cookie automaticamente para o mesmo site e o ASP.NET Core reconstrói a identidade do usuário a partir dele.

Quando o usuário faz login, o servidor responde ao navegador com um cookie:

```
Set-Cookie: .AspNetCore.Identity.Application=CfDJ8HqL8X9aQ3...mZp2w
```

Depois disso, em cada nova requisição para o mesmo site, o navegador envia automaticamente:

```
Cookie: .AspNetCore.Identity.Application=CfDJ8HqL8X9aQ3...mZp2w
```

ASP.NET Core lê esse cookie e descobre internamente:

Usuário: Ana

Id: 42

Unidade III

Role: Tutor

Está autenticada: Sim

Já o JWT, ou JSON Web Token, representa outro modelo de transporte de identidade. Um JWT é um token textual, geralmente enviado no cabeçalho Authorization, muito comum em APIs consumidas por clientes independentes, aplicações móveis, integrações entre serviços e front-ends distribuídos separadamente.

Cookies e JWT podem participar de arquiteturas seguras, mas respondem a contextos diferentes. Neste subtópico, o PetMatch usa cookies com ASP.NET Identity, porque o fluxo prático envolve navegador, páginas MVC protegidas e integração com a experiência React por meio do mesmo back-end.



Saiba mais

Uma referência fundamental sobre cookies HTTP, definindo os cabeçalhos Cookie e Set-Cookie e explicando seu papel na manutenção de estado em um protocolo originalmente sem estado, é:

BARTH, A. *HTTP State Management Mechanism*. IETF, 2011. Disponível em: <https://tinyurl.com/7yu7yjrj>. Acesso em: 8 jul. 2026.

Já a especificação clássica do JWT, útil para compreender claims, assinatura, representação compacta de tokens e uso em autenticação e autorização em APIs, é:

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. *JSON Web Token (JWT)*. IETF, 2015. RFC 7519. Disponível em: <https://tinyurl.com/3zfbe9uw>. Acesso em: 8 jul. 2026.

Roles e claims expressam autorização em níveis diferentes. Uma role representa um papel amplo do usuário no sistema, como Voluntario ou Administrador.

Uma claim representa uma afirmação sobre o usuário, como nome, cidade, identificador, permissão específica ou vínculo institucional.

O usuário pode abrir a home, filtrar pets, navegar pela lista e acessar detalhes sem criar conta. As ações de cadastro e edição, por outro lado, passam a exigir autenticação e role adequada, pois alteram informações do sistema. Ao tentar cadastrar ou editar sem entrar, o usuário é levado à tela de login.

No projeto PetMatch.Web, o arquivo Models/ApplicationUser.cs define o usuário da aplicação. Ele deriva de IdentityUser, recebendo toda a infraestrutura de Identity e acrescentando dados próprios do PetMatch.

```
using Microsoft.AspNetCore.Identity;

namespace PetMatch.Web.Models;

public sealed class ApplicationUser : IdentityUser
{
    public string NomeCompleto { get; set; } = string.Empty;

    public string Cidade { get; set; } = string.Empty;
}
```

Esse modelo permite que o usuário tenha e-mail, senha protegida e identificador fornecidos pelo Identity, além de informações de perfil usadas pela aplicação. NomeCompleto e Cidade tornam a tela de perfil mais concreta e mostram que Identity pode ser estendido com propriedades do domínio.

```
namespace PetMatch.Web.Models;

public static class PetMatchRoles
{
    public const string Administrador = "Administrador";

    public const string Voluntario = "Voluntario";
}
```

Para armazenar usuários e roles no mesmo banco SQLite, o contexto passa a herdar de `IdentityDbContext`. `Data/PetMatchDbContext.cs`:

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using PetMatch.Web.Models;
```

50

DESENVOLVIMENTO WEB COM .NET

```
namespace PetMatch.Web.Data;

public sealed class PetMatchDbContext : IdentityDbContext<ApplicationUser>
{
    public PetMatchDbContext(DbContextOptions<PetMatchDbContext> options)
        : base(options)
    {
    }

    public DbSet<Pet> Pets => Set<Pet>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Pet>(entity =>
        {
            entity.HasKey(pet => pet.Id);
            entity.Property(pet => pet.Nome).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.Especie).HasMaxLength(40).IsRequired();
            entity.Property(pet => pet.Porte).HasMaxLength(30).IsRequired();
            entity.Property(pet => pet.Cidade).HasMaxLength(80).IsRequired();
            entity.Property(pet => pet.RowVersion).IsConcurrencyToken().
IsRequired();
        });
    }
}
```

O registro dos serviços de autenticação fica em `Program.cs`, no projeto `PetMatch.Web`. Esse trecho configura `Identity`, banco, roles, cookie e caminhos das páginas de conta.

```
builder.Services
    .AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.User.RequireUniqueEmail = true;
        options.SignIn.RequireConfirmedAccount = false;
    })
    .AddEntityFrameworkStores<PetMatchDbContext>()
    .AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.Cookie.Name = "PetMatch.Auth";
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
});
```

O seed da aplicação foi ampliado para preparar roles, sem gravar uma senha fixa no código. No projeto PetMatch.Web, o arquivo Data/PetMatchSeed.cs cria Administrador e Voluntario, preservando também o seed dos pets já existentes.

```
public static async Task SeedRolesAsync(RoleManager<IdentityRole> roleManager)
{
    var roles = new[]
    {
        PetMatchRoles.Administrador,
        PetMatchRoles.Voluntario
    };

    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
        {
            await roleManager.CreateAsync(new IdentityRole(role));
        }
    }
}

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (!ModelState.IsValid)
    {
        return View(model);
    }

    var user = new ApplicationUser
    {
        UserName = model.Email,
        Email = model.Email,
        NomeCompleto = model.NomeCompleto,
        Cidade = model.Cidade
    };

    var result = await _userManager.CreateAsync(user, model.Senha);

    if (!result.Succeeded)
    {
        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }

        return View(model);
    }

    await _userManager.AddToRoleAsync(user, PetMatchRoles.Voluntario);
    await _signInManager.SignInAsync(user, isPersistent: false);

    return RedirectToAction(nameof(Profile));
}
```

A proteção das páginas de cadastro e edição de pets fica declarada no controller responsável pelos pets. No projeto PetMatch.Web, o arquivo Controllers/PetsController.cs protege as actions sensíveis com [Authorize].

```
[Authorize(Roles = $"{PetMatchRoles.Voluntario},{PetMatchRoles.Administrador}")]  
[HttpGet]  
public IActionResult Create()  
{  
    return View(new PetCreateViewModel());  
}  
  
[Authorize(Roles = $"{PetMatchRoles.Voluntario},{PetMatchRoles.Administrador}")]  
[HttpGet]  
public async Task<IActionResult> Edit(int id)  
{  
    // Carrega o pet e exibe o formulário de edição.  
}
```

A tela de login fica em Views/Account/Login.cshtml, no projeto PetMatch.Web. Ela usa Tag Helpers e o LoginViewModel para capturar e-mail, senha e retorno visual de validação.

```
@model LoginViewModel  
  
{  
    ViewData["Title"] = "Entrar no PetMatch";  
}  
  
<section class="account-panel">  
    <h1>Entrar no PetMatch</h1>  
    <p>Acesse sua conta para cadastrar e gerenciar pets disponíveis para  
    adoção.</p>  
  
    <form asp-action="Login" method="post" class="account-form">  
        <div asp-validation-summary="ModelOnly" class="validation-summary"></div>  
  
        <label asp-for="Email"></label>  
        <input asp-for="Email" autocomplete="email" />  
        <span asp-validation-for="Email" class="field-validation"></span>  
  
        <label asp-for="Senha"></label>  
        <input asp-for="Senha" type="password" autocomplete="current-password"  
    />  
        <span asp-validation-for="Senha" class="field-validation"></span>  
  
        <button type="submit">Entrar</button>  
    </form>  
  
    <a asp-controller="Account" asp-action="Register">Criar conta</a>  
</section>
```

A página de perfil usa `ProfileViewModel` e exibe dados do usuário autenticado. Ela também pode mostrar roles e claims principais de forma compreensível, ajudando o leitor a perceber como informações de identidade aparecem na aplicação.

```
@model ProfileViewModel

@{
    ViewData["Title"] = "Meu perfil";
}
```

54

DESENVOLVIMENTO WEB COM .NET

```
<section class="account-panel">
    <h1>Meu perfil</h1>

    <dl class="profile-list">
        <dt>Nome</dt>
        <dd>@Model.NomeCompleto</dd>

        <dt>E-mail</dt>
        <dd>@Model.Email</dd>

        <dt>Cidade</dt>
        <dd>@Model.Cidade</dd>

        <dt>Perfil de acesso</dt>
        <dd>@Model.Role</dd>
    </dl>

    <form asp-controller="Account" asp-action="Logout" method="post">
        <button type="submit">Sair</button>
    </form>
</section>
```

6.2 SPA – login/logout, armazenamento seguro de tokens e rotas protegidas

A primeira tarefa do front-end é descobrir esse estado: há usuário autenticado, a sessão expirou, a chamada está carregando ou o visitante está anônimo.

O armazenamento seguro de tokens é um dos pontos mais sensíveis em aplicações no navegador. Quando uma aplicação usa JWT explícito, o front-end recebe um token e precisa decidir como mantê-lo entre requisições. Armazenamentos acessíveis por JavaScript, como `localStorage` e `sessionStorage`, ampliam a exposição do token se houver execução indevida de scripts na página. Por isso,

aplicações web que usam o navegador como cliente principal frequentemente preferem um cookie protegido emitido pelo servidor, com atributos como `HttpOnly`, `Secure` e `SameSite`. Nesse modelo, o JavaScript não lê o token, o navegador envia o cookie nas requisições e o servidor valida a sessão.

No fluxo implementado aqui, o PetMatch usa o caminho já preparado no back-end: ASP.NET Identity com cookie de autenticação. A SPA React interage com esse cookie por meio de chamadas HTTP com credentials: “include”, mantendo o token fora do alcance direto do código JavaScript.

```
namespace PetMatch.Web.Contracts;

public sealed record AuthLoginRequest(
    string Email,
    string Senha,
    string? ReturnUrl);

public sealed record AuthRegisterRequest(
    string NomeCompleto,
    string Cidade,
    string Email,
    string Senha,
    string? ReturnUrl);

public sealed record AuthUserResponse(
    bool IsAuthenticated,
    string? Email,
    string? NomeCompleto,
    IReadOnlyList<string> Roles,
    IReadOnlyList<string> Claims);
```

Esses contratos tornam explícito o diálogo entre React e ASP.NET Core. A SPA envia e-mail e senha para login, envia dados de cadastro para criar conta e recebe um objeto de sessão que informa se há usuário autenticado, quais roles ele possui e quais claims principais estão disponíveis.

Os endpoints de autenticação ficam em `Endpoints/AuthEndpoints.cs`, no projeto `PetMatch.Web`. Eles usam os serviços já configurados do ASP.NET Identity e publicam operações JSON para a SPA.

```
using Microsoft.AspNetCore.Identity;
using PetMatch.Web.Contracts;
using PetMatch.Web.Models;

namespace PetMatch.Web.Endpoints;

public static class AuthEndpoints
{
    public static IEndpointRouteBuilder MapAuthEndpoints(this
        IEndpointRouteBuilder endpoints)
    {
        -
```

```

namespace PetMatch.Web.Contracts;

public sealed record AuthLoginRequest(
    string Email,
    string Senha,
    string? returnUrl);

public sealed record AuthRegisterRequest(
    string NomeCompleto,
    string Cidade,
    string Email,
    string Senha,
    string? returnUrl);

public sealed record AuthUserResponse(
    bool IsAuthenticated,
    string? Email,
    string? NomeCompleto,
    IReadOnlyList<string> Roles,
    IReadOnlyList<string> Claims);

```

Esses contratos tornam explícito o diálogo entre React e ASP.NET Core. A SPA envia e-mail e senha para login, envia dados de cadastro para criar conta e recebe um objeto de sessão que informa se há usuário autenticado, quais roles ele possui e quais claims principais estão disponíveis.

Os endpoints de autenticação ficam em `Endpoints/AuthEndpoints.cs`, no projeto `PetMatch.Web`. Eles usam os serviços já configurados do ASP.NET Identity e publicam operações JSON para a SPA.

```

using Microsoft.AspNetCore.Identity;
using PetMatch.Web.Contracts;
using PetMatch.Web.Models;

namespace PetMatch.Web.Endpoints;

public static class AuthEndpoints
{
    public static IEndpointRouteBuilder MapAuthEndpoints(this
        IEndpointRouteBuilder endpoints)
    {

```

```
builder.Services.ConfigureApplicationCookie(options =>
{
    options.Cookie.Name = "PetMatch.Auth";
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
```

Unidade III

```
options.Events.OnRedirectToLogin = context =>
{
    if (context.Request.Path.StartsWithSegments("/api"))
    {
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;
        return Task.CompletedTask;
    }

    context.Response.Redirect(context.RedirectUri);
    return Task.CompletedTask;
};

options.Events.OnRedirectToAccessDenied = context =>
{
    if (context.Request.Path.StartsWithSegments("/api"))
    {
        context.Response.StatusCode = StatusCodes.Status403Forbidden;
        return Task.CompletedTask;
    }

    context.Response.Redirect(context.RedirectUri);
    return Task.CompletedTask;
};
});
```

No front-end, o cliente de autenticação fica em `src/api/authClient.ts`, no projeto `PetMatch`. Frontend. Ele usa `fetch` com `credentials: "include"` para enviar e receber o cookie de autenticação.

```
export type AuthUser = {
  isAuthenticated: boolean;
  email: string | null;
  nomeCompleto: string | null;
  roles: string[];
  claims: string[];
};

export type LoginRequest = {
  email: string;
  senha: string;
  returnUrl?: string;
};

export async function getCurrentUser(): Promise<AuthUser> {
  const response = await fetch('/api/auth/me', {
    credentials: 'include'
  });

  if (!response.ok) {
    throw new Error('Não foi possível verificar sua sessão.');
```

```
  }

  return response.json();
}

export async function login(request: LoginRequest): Promise<AuthUser> {
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify(request)
  });

  if (!response.ok) {
    throw new Error('E-mail ou senha inválidos.');
```

```
  }

  return response.json();
}

export async function logout(): Promise<void> {
  await fetch('/api/auth/logout', {
    method: 'POST',
    credentials: 'include'
  });
}
```

```

import { createContext, useContext, useEffect, useMemo, useState } from 'react';
import { AuthUser, getCurrentUser, login as loginRequest, logout as
logoutRequest } from '../api/authClient';

type AuthContextValue = {
  user: AuthUser | null;
  loading: boolean;
  isAuthenticated: boolean;
  login: (email: string, senha: string, returnUrl?: string) => Promise<void>;
  logout: () => Promise<void>;
  refreshUser: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(null);
  const [loading, setLoading] = useState(true);

```

Unidade III

```

async function refreshUser() {
  const currentUser = await getCurrentUser();
  setUser(currentUser);
}

async function login(email: string, senha: string, returnUrl?: string) {
  const authenticatedUser = await loginRequest({ email, senha, returnUrl });
  setUser(authenticatedUser);
}

```

```

    async function logout() {
      await logoutRequest();
      setUser({ isAuthenticated: false, email: null, nomeCompleto: null, roles:
[], claims: [] });
    }

    useEffect(() => {
      refreshUser().finally(() => setLoading(false));
    }, []);

    const value = useMemo(() => ({
      user,
      loading,
      isAuthenticated: user?.isAuthenticated === true,
      login,
      logout,
      refreshUser
    }), [user, loading]);

    return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
  }

  export function useAuth() {
    const context = useContext(AuthContext);

    if (context === null) {
      throw new Error('useAuth deve ser usado dentro de AuthProvider.');
```

O arquivo `src/main.tsx`, no projeto `PetMatch.Frontend`, passa a envolver a aplicação com `AuthProvider`, mantendo o `BrowserRouter` já usado pelo roteamento.

```

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>
);
```

```

import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../auth/AuthContext';
import { RequestStatus } from '../RequestStatus';

export function ProtectedRoute() {
  const { isAuthenticated, loading } = useAuth();
  const location = useLocation();

  if (loading) {
    return <RequestStatus title="Verificando acesso" description="Estamos confirmando sua sessão." />;
  }

  if (!isAuthenticated) {
    return <Navigate to="/entrar" state={{ returnUrl: location.pathname }} replace />;
  }

  return <Outlet />;
}

```

Esse componente melhora a navegação da SPA. A rota `/perfil` fica protegida na experiência React e a tentativa de acesso anônimo leva à tela de entrada. A proteção de dados continua no servidor e o front-end organiza a experiência antes que o usuário chegue a uma tela inadequada.

As rotas da aplicação são declaradas em `src/App.tsx`, no projeto `PetMatch.Frontend`. As páginas públicas permanecem acessíveis e o perfil fica dentro da rota protegida.

```

import { Route, Routes } from 'react-router-dom';
import { AppNav } from '../components/AppNav';
import { ProtectedRoute } from '../components/ProtectedRoute';
import { LoginPage } from '../pages/LoginPage';
import { PetDetailsPage } from '../pages/PetDetailsPage';
import { PetListPage } from '../pages/PetListPage';
import { ProfilePage } from '../pages/ProfilePage';
import { RegisterPage } from '../pages/RegisterPage';

```

```

export function App() {
  return (
    <>
      <AppNav />

      <Routes>
        <Route path="/" element={<PetListPage />} />
        <Route path="/pets/:id" element={<PetDetailsPage />} />
        <Route path="/entrar" element={<LoginPage />} />
        <Route path="/criar-conta" element={<RegisterPage />} />

        <Route element={<ProtectedRoute />>
          <Route path="/perfil" element={<ProfilePage />} />
        </Route>
      </Routes>
    </>
  );
}

```

```

import { FormEvent, useState } from 'react';
import { useLocation, useNavigate } from 'react-router-dom';
import { useAuth } from '../auth/AuthContext';

export function LoginPage() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');
  const [error, setError] = useState('');

  async function handleSubmit(event: FormEvent) {
    event.preventDefault();
    setError('');

    try {
      const returnUrl = location.state?.returnUrl ?? '/perfil';
      await login(email, senha, returnUrl);
      navigate(returnUrl);
    } catch {
      setError('Não foi possível entrar com os dados informados.');
```

```

    }
  }

  return (
    <main className="account-shell">
      <section className="account-card">
        <h1>Entrar no PetMatch</h1>
        <p>Acesse sua conta para acompanhar seu perfil e gerenciar cadastros
        autorizados.</p>

```

64

DESENVOLVIMENTO WEB COM .NET

```

    <form onSubmit={handleSubmit} className="account-form">
      {error && <p className="status-card status-error">{error}</p>}

      <label htmlFor="email">E-mail</label>
      <input id="email" value={email} onChange={event => setEmail(event.
target.value)} />

      <label htmlFor="senha">Senha</label>
      <input id="senha" type="password" value={senha} onChange={event =>
setSenha(event.target.value)} />

      <button type="submit">Entrar</button>
    </form>
  </section>
</main>
);
}

```

Unidade IV

7 OBSERVABILIDADE E RESILIÊNCIA

Quando passa a ser executada em um ambiente real, ela fica sujeita a variações de carga, falhas de rede, indisponibilidade temporária de serviços externos, erros de configuração, dados inesperados, lentidão em consultas, exceções não previstas e comportamentos que nem sempre podem ser reproduzidos facilmente em desenvolvimento. Por isso, além de projetar funcionalidades, é necessário projetar a capacidade de compreender a aplicação em execução e de mantê-la operante diante de condições adversas.

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir dos sinais que ele produz durante sua execução. Em aplicações ASP.NET Core, esses sinais podem incluir registros de eventos, métricas, rastreamentos distribuídos, informações sobre requisições, exceções, tempos de resposta, consumo de recursos e resultados de operações críticas.

Se uma pessoa interessada em adoção envia um formulário e a operação falha, é necessário compreender se o problema ocorreu na validação, na persistência, na comunicação com outro serviço ou na infraestrutura. Se uma organização tenta cadastrar um pet e a página responde lentamente, é preciso identificar se a causa está no banco de dados, na quantidade de dados carregados, em uma dependência externa ou no próprio servidor. Se determinados usuários relatam dificuldade para acessar uma área protegida, a equipe precisa investigar o fluxo de autenticação, autorização e sessão. Sem sinais adequados, a aplicação torna-se opaca; com observabilidade, ela passa a oferecer evidências para diagnóstico.

Os registros de eventos, frequentemente chamados de logs, constituem uma das formas mais conhecidas de observabilidade. Eles permitem narrar acontecimentos relevantes da aplicação, como início e término de operações, falhas inesperadas, tentativas de acesso negado, exceções tratadas e mudanças importantes de estado. No entanto, logs eficazes exigem critério. Informações excessivas podem gerar ruído e dificultar a investigação; informações insuficientes podem tornar um problema invisível. Além disso, registros devem respeitar a privacidade e a segurança dos usuários, evitando exposição indevida de dados sensíveis.

As métricas oferecem outra perspectiva. Enquanto logs descrevem eventos específicos, métricas permitem acompanhar tendências e quantificar comportamentos ao longo do tempo. Taxa de requisições, duração média de respostas, quantidade de erros, uso de memória, disponibilidade de endpoints e desempenho de operações de banco de dados são exemplos de sinais que ajudam a perceber a saúde geral da aplicação.

O rastreamento, por sua vez, ajuda a acompanhar o percurso de uma requisição por meio de diferentes partes da aplicação e, em arquiteturas distribuídas, por diferentes serviços. Ele permite compreender como uma solicitação se desloca entre endpoint, regras de aplicação, acesso ao banco, chamadas externas e geração de resposta. Essa visão é importante quando um problema não está concentrado em um único ponto evidente. Uma página lenta pode ser consequência de uma consulta ineficiente, de uma chamada externa demorada ou de várias pequenas operações acumuladas, ajuda a reconstruir o caminho da requisição e a localizar onde o tempo foi gasto ou onde a falha ocorreu.

A resiliência, embora relacionada à operação, possui foco diferente. Ela expressa a capacidade da aplicação de continuar oferecendo comportamento adequado mesmo quando algo falha ou demora mais do que o esperado.

No PetMatch, a resiliência pode ser percebida na forma como a aplicação lida com situações adversas sem abandonar o usuário diante de mensagens incompreensíveis ou estados inconsistentes. Se uma consulta temporariamente não puder recuperar a lista de pets disponíveis, a aplicação deve responder de modo controlado, preservando a navegação e comunicando a situação adequadamente.

Estratégias de resiliência podem incluir tratamento adequado de exceções, definição de tempos limite, repetição controlada de operações transitórias, isolamento de falhas, degradação graciosa e proteção contra sobrecarga.

7.1 Logging, métricas, tracing e Health Checks

A observabilidade é a capacidade de compreender o comportamento de uma aplicação enquanto ela está em execução.

O logging é o registro de eventos relevantes produzidos pela aplicação. No ASP.NET Core, logs são integrados à infraestrutura da plataforma e podem ser emitidos por controllers, endpoints, serviços e pelo próprio pipeline HTTP. Um log de nível

Information descreve uma operação esperada, como a listagem de pets. Um log de nível Warning indica uma situação que merece atenção, como uma tentativa de login inválida. Um log de nível Error registra uma falha que impediu a conclusão de uma operação (valor do logging: a mensagem deve informar o evento, seus dados operacionais relevantes e o ponto da aplicação em que ocorreu).

Logs estruturados são preferíveis a mensagens montadas manualmente porque preservam os valores como propriedades nomeadas. Em vez de concatenar textos, o código registra campos como página consultada, tamanho da página, identificador do pet ou resultado da operação.

Em fluxos de autenticação, o cuidado é ainda maior: a aplicação pode registrar que uma tentativa de login falhou, mas não deve registrar senha, cookie, token ou qualquer segredo.

As métricas representam medições agregadas do comportamento da aplicação. Enquanto um log descreve um evento específico, uma métrica mostra volume, frequência ou tendência. Um contador pode indicar quantas vezes a listagem de pets foi consultada, quantas páginas de detalhe foram acessadas ou quantas tentativas de login foram inválidas. Essa diferença é fundamental: logs ajudam a investigar eventos e métricas ajudam a perceber padrões. No .NET, System.Diagnostics.Metrics permite criar instrumentos como contadores sem adicionar pacotes externos.

Já o tracing acompanha o percurso de uma operação. Um trace permite demarcar uma atividade e relacioná-la ao fluxo de execução que está ocorrendo. Em uma requisição de listagem, por exemplo, a atividade pode envolver a chegada ao endpoint, a aplicação de filtros, a consulta ao banco e a geração da resposta. No .NET, ActivitySource é o mecanismo usado para criar essas atividades.

As Health Checks são verificações objetivas de saúde da aplicação. Uma verificação de vida responde se o processo está ativo e consegue responder. Uma verificação de prontidão responde se a aplicação está em condição de atender adequadamente, considerando dependências essenciais. No PetMatch, essa distinção aparece em dois endpoints: /health/live, que confirma que a aplicação está viva (figura 49), e /health/ready, que verifica também a prontidão do SQLite por meio do PetMatchDbContext.

Para organizar o tracing, foi criada a pasta Observability no projeto PetMatch.Web. O arquivo `PetMatch.Web/Observability/PetMatchDiagnostics.cs` concentra o `ActivitySource` usado pela aplicação.

```
using System.Diagnostics;

namespace PetMatch.Web.Observability;

public static class PetMatchDiagnostics
{
    public const string ActivitySourceName = "PetMatch.Web";

    public static readonly ActivitySource ActivitySource =
        new(ActivitySourceName);
}
```

Esse trecho cria uma origem de tracing chamada `PetMatch.Web`. Endpoints e controllers podem iniciar atividades a partir dela, mantendo consistência nos nomes e evitando strings repetidas em vários arquivos.

As métricas ficam centralizadas em `PetMatch.Web/Observability/PetMatchMetrics.cs`.

```
using System.Diagnostics.Metrics;

namespace PetMatch.Web.Observability;

public static class PetMatchMetrics
{
    public const string MeterName = "PetMatch.Web";

    private static readonly Meter Meter = new(MeterName);

    public static readonly Counter<long> PetListRequests =
        Meter.CreateCounter<long>("petmatch.pet_list_requests");

    public static readonly Counter<long> PetDetailsRequests =
        Meter.CreateCounter<long>("petmatch.pet_details_requests");

    public static readonly Counter<long> SuccessfulLogins =
        Meter.CreateCounter<long>("petmatch.successful_logins");

    public static readonly Counter<long> InvalidLogins =
        Meter.CreateCounter<long>("petmatch.invalid_logins");
}
```

Esse código define um `Meter` e quatro contadores. A aplicação pode incrementar esses contadores quando a listagem de pets é acessada, quando um detalhe é

consultado, quando um login é bem-sucedido ou quando uma tentativa inválida ocorre.

A verificação de prontidão do banco foi criada em PetMatch.Web/HealthChecks/PetMatchDatabaseHealthCheck.cs. Esse Health Check usa o PetMatchDbContext para consultar o SQLite e confirmar que a persistência está acessível.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Diagnostics.HealthChecks;
using PetMatch.Web.Data;

namespace PetMatch.Web.HealthChecks;

public sealed class PetMatchDatabaseHealthCheck(PetMatchDbContext dbContext)
    : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        try
        {
            var petsCadastrados = await
dbContext.Pets.CountAsync(cancellationToken);

            return HealthCheckResult.Healthy(
                "Banco de dados acessível.",
                new Dictionary<string, object>
                {
                    ["petsCadastrados"] = petsCadastrados
                });
        }
        catch (Exception exception)
        {
            return HealthCheckResult.Unhealthy(
                "Não foi possível consultar o banco de dados.",
                exception);
        }
    }
}
```

O registro dos Health Checks e o mapeamento dos endpoints ficam em PetMatch.Web/[Program.cs](#):

```
builder.Services
    .AddHealthChecks()
    .AddCheck<PetMatchDatabaseHealthCheck>("sqlite");

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
```

Unidade IV

```
    Predicate = _ => false
});

app.MapHealthChecks("/health/ready");
```

Os endpoints de pets também passam a emitir sinais de observabilidade. Em PetMatch.Web/Endpoints/PetEndpoints.cs, a consulta de listagem pode iniciar uma atividade, registrar um log estruturado e incrementar uma métrica.

```
using var activity = PetMatchDiagnostics.ActivitySource.StartActivity("Listar
pets");

logger.LogInformation(
    "Listagem de pets solicitada. Página: {Page}; Tamanho: {PageSize}",
    query.Page,
    query.PageSize);

PetMatchMetrics.PetListRequests.Add(1);
```

ENTÃO WEB COM .NET Nos fluxos de autenticação, PetMatch.Web/Endpoints/AuthEndpoints.cs e PetMatch.Web/Controllers/AccountController.cs registram eventos sem expor dados sensíveis.

```
logger.LogWarning(
    "Tentativa de login inválida para o e-mail {Email}.",
    request.Email);

PetMatchMetrics.InvalidLogins.Add(1);
```

Nos fluxos MVC, PetMatch.Web/Controllers/PetsController.cs registra eventos de cadastro e edição.

```

_logger.LogInformation(
    "Pet cadastrado com sucesso. PetId: {PetId}; Nome: {Nome}",
    pet.Id,
    pet.Nome);

```

Os níveis de log são configurados nos arquivos `PetMatch.Web/appsettings.json` e `PetMatch.Web/appsettings.Development.json`. Em desenvolvimento, é útil manter os logs da aplicação visíveis e reduzir ruídos de infraestrutura.

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Warning",
      "PetMatch.Web": "Information"
    }
  }
}

```

7.2 Políticas de resiliência (retry/circuit breaker) e rate limiting/output caching (quando disponíveis)

O `retry` é a repetição controlada de uma operação que falhou por uma condição transitória. A palavra “controlada” é decisiva: uma política de `retry` precisa definir quantidade máxima de tentativas, intervalo entre elas e tipo de falha que justifica nova tentativa. Esse mecanismo é adequado para operações de leitura ou chamadas idempotentes, nas quais repetir a operação não provoca efeito duplicado. Operações como cadastro, edição, login ou qualquer ação com efeito de gravação exigem mais cautela, pois a repetição automática pode produzir resultados indesejados.

O `circuit breaker` é uma política de proteção aplicada quando uma dependência começa a falhar repetidamente. Se a aplicação insiste em chamar um recurso que já demonstrou instabilidade, ela consome conexões, threads, tempo de resposta e capacidade de processamento. O `circuit breaker` observa falhas sucessivas e, ao atingir determinado limite, abre o circuito por um período. Enquanto o circuito está aberto, a aplicação evita novas chamadas e usa uma resposta alternativa. Depois, permite uma nova tentativa.

O rate limiting é a limitação do volume de requisições aceitas em determinado intervalo. Ele protege a aplicação contra uso excessivo, automações agressivas, erros de cliente e abuso de endpoints sensíveis. Uma API pública de leitura pode ter limites mais generosos, enquanto login e cadastro devem receber limites mais restritivos. Quando o limite é excedido, o status HTTP adequado é 429 Too Many Requests, indicando que o cliente fez chamadas demais em curto período.

Já o output caching é o armazenamento temporário da resposta produzida pela aplicação. Quando uma resposta é pública, segura e independente do usuário atual, ela pode ser reaproveitada por um curto período, reduzindo consultas, processamento e tempo de resposta. O cuidado principal está no escopo: páginas de perfil, login, áreas autenticadas ou respostas personalizadas por cookie, claims ou sessão não devem ser armazenadas como cache compartilhado.

Para centralizar os nomes das políticas, foi criada a pasta Resilience no projeto PetMatch.Web. O arquivo PetMatch.Web/Resilience/PetMatchResiliencePolicies.cs reúne constantes usadas por rate limiting, output caching e resiliência.

```
namespace PetMatch.Web.Resilience;

public static class PetMatchResiliencePolicies
{
    public const string PublicApiRateLimit = "petmatch-public-api";
    public const string AuthRateLimit = "petmatch-auth";
    public const string PublicPetsCache = "petmatch-public-pets";
    public const string AdoptionTipsCache = "petmatch-adoption-tips";
}
```

A configuração central das políticas fica em PetMatch.Web/Program.cs.

```

builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter(PetMatchResiliencePolicies.PublicApiRateLimit,
limiterOptions =>
    {
        limiterOptions.PermitLimit = 6;
        limiterOptions.Window = TimeSpan.FromSeconds(30);
    });

    options.AddFixedWindowLimiter(PetMatchResiliencePolicies.AuthRateLimit,
limiterOptions =>
    {

```

34

DESENVOLVIMENTO WEB COM .NET

```

        limiterOptions.PermitLimit = 4;
        limiterOptions.Window = TimeSpan.FromMinutes(1);
    });
});

builder.Services.AddOutputCache(options =>
{
    options.AddPolicy(PetMatchResiliencePolicies.PublicPetsCache, builder =>
        builder.Expire(TimeSpan.FromSeconds(20)));

    options.AddPolicy(PetMatchResiliencePolicies.AdoptionTipsCache, builder =>
        builder.Expire(TimeSpan.FromSeconds(30)));
});

```

Esse código cria dois limites de requisição e duas políticas de cache. Os valores são curtos para fins didáticos, permitindo que o leitor observe o comportamento no navegador e no Output do Visual Studio. A política pública aceita chamadas de leitura em volume moderado e a política de autenticação é mais restritiva. O cache de pets e dicas dura poucos segundos, reduzindo repetição de processamento sem mascarar mudanças por muito tempo.

Depois de registrar os serviços, os middlewares precisam entrar no pipeline da aplicação. Ainda em `PetMatch.Web/Program.cs`, as chamadas a seguir ativam os recursos durante o processamento das requisições.

```

app.UseRateLimiter();

app.UseOutputCache();

```

Essas chamadas fazem com que as políticas aplicadas aos endpoints tenham efeito real. Sem elas, as configurações existiriam nos serviços, mas não influenciariam as respostas HTTP.

A demonstração de retry e circuit breaker foi implementada de forma didática, sem Microsoft.Extensions.Http.Resilience. O arquivo PetMatch.Web/Services/AdoptionTipClient.cs concentra a chamada de leitura que busca uma dica de adoção responsável.

```
public async Task<AdoptionTipResponse> GetTipAsync(CancellationToken
cancellationToken)
{
    if (_circuitState.IsOpen)
    {
        return AdoptionTipResponse.Fallback();
    }

    for (var attempt = 1; attempt <= 3; attempt++)
    {
        try
        {
```

Unidade IV

```
        var response = await _httpClient.GetFromJsonAsync<AdoptionTipRespon
se>(
            "/api/dicas/adocao-parceiro",
            cancellationToken);

        _circuitState.ReportSuccess();

        return response ?? AdoptionTipResponse.Fallback();
    }
    catch (Exception exception) when (attempt < 3)
    {
        _logger.LogWarning(
            exception,
            "Falha transitória ao buscar dica de adoção. Tentativa
{Attempt}.",
            attempt);
    }

    _circuitState.ReportFailure();

    return AdoptionTipResponse.Fallback();
}
```

A aplicação tenta obter a dica, registra falhas temporárias, informa sucesso ou falha ao circuito e devolve uma resposta alternativa quando necessário. O usuário não vê uma tela quebrada; ele recebe uma orientação útil mesmo quando a dependência da dica não responde como esperado. A resposta usada pela dica foi criada em PetMatch.Web/Services/AdoptionTipResponse.cs.

```
namespace PetMatch.Web.Services;

public sealed record AdoptionTipResponse(
    string Titulo,
    string Mensagem)
{
    public static AdoptionTipResponse Fallback() =>
        new(
            "Cuidado responsável",
            "Antes da adoção, confirme se sua rotina comporta tempo, espaço e acompanhamento veterinário.");
}
```

Esse código cria uma resposta com título e mensagem e define uma alternativa segura. A consequência prática é que a view MVC pode sempre exibir uma dica coerente, mesmo quando o cliente resiliente precisa acionar fallback. O endpoint público da dica foi criado em `PetMatch.Web/Endpoints/AdoptionTipEndpoints.cs`. Ele publica GET `/api/dicas/adocao-parceiro`, retornando uma orientação em JSON.

```
app.MapGet("/api/dicas/adocao-parceiro", () =>
    Results.Ok(new AdoptionTipResponse(
```

36

DESENVOLVIMENTO WEB COM .NET

```
        "Adaptação gradual",
        "Nos primeiros dias, mantenha água, descanso e uma rotina tranquila para o pet se sentir seguro.")))
    .CacheOutput(PetMatchResiliencePolicies.AdoptionTipsCache)
    .RequireRateLimiting(PetMatchResiliencePolicies.PublicApiRateLimit);
```

Os endpoints públicos de pets foram ajustados em `PetMatch.Web/Endpoints/PetEndpoints.cs`.

```
group.MapGet("/", ListPetsAsync)
    .CacheOutput(PetMatchResiliencePolicies.PublicPetsCache)
    .RequireRateLimiting(PetMatchResiliencePolicies.PublicApiRateLimit);

group.MapGet("/{id:int}", GetPetByIdAsync)
    .CacheOutput(PetMatchResiliencePolicies.PublicPetsCache)
    .RequireRateLimiting(PetMatchResiliencePolicies.PublicApiRateLimit);
```

Os endpoints de autenticação foram ajustados em `PetMatch.Web/Endpoints/AuthEndpoints.cs`. Login e cadastro recebem uma política mais restritiva, pois são fluxos sensíveis.

```
auth.MapPost("/login", LoginAsync)
    .RequireRateLimiting(PetMatchResiliencePolicies.AuthRateLimit);

auth.MapPost("/register", RegisterAsync)
    .RequireRateLimiting(PetMatchResiliencePolicies.AuthRateLimit);
```

A experiência visual aparece em /Pets. O arquivo `PetMatch.Web/Controllers/PetsController.cs` consome o `AdoptionTipClient` e envia a dica para a view. A página `PetMatch.Web/Views/Pets/Index.cshtml` exibe uma faixa "cuidado responsável" junto à listagem.

```
var adoptionTip = await _adoptionTipClient.GetTipAsync(cancellationToken);  
ViewData["AdoptionTipTitle"] = adoptionTip.Titulo;  
ViewData["AdoptionTipMessage"] = adoptionTip.Mensagem;
```

8 BUILD, IMPLANTAÇÃO E CI/CD FULL-STACK

O build é a etapa em que o código-fonte, as dependências, os arquivos de configuração e os recursos da aplicação são processados para gerar um resultado executável ou publicável. Em aplicações .NET, isso envolve compilar o código C#, verificar referências, preparar arquivos estáticos, aplicar configurações de projeto e produzir artefatos que possam ser executados em um ambiente de destino. Em uma aplicação full-stack, a ideia de build precisa considerar também o front-end, pois interfaces construídas com tecnologias como React passam por seu próprio processo de preparação, no qual componentes, módulos, folhas de estilo, imagens e scripts são transformados em arquivos otimizados para o navegador.

A implantação, por sua vez, é o processo de disponibilizar a aplicação em um ambiente no qual ela possa ser acessada e utilizada. Para publicar um sistema é necessário considerar configurações de ambiente, variáveis sensíveis, conexão com banco de dados, endereços de APIs, políticas de segurança, certificados, permissões, arquivos estáticos, logs, monitoramento e compatibilidade entre os componentes instalados.

A API precisa estar publicada em endereço acessível e configurada para aceitar chamadas da interface autorizada. O front-end precisa conhecer corretamente os endereços dos serviços que consome. O banco de dados precisa estar disponível, inicializado e protegido. As configurações de autenticação, autorização e comunicação entre camadas precisam permanecer coerentes. O ciclo de build e implantação deve, portanto, refletir a integração real do sistema.

O CI/CD amplia essa visão ao propor que integração e entrega sejam apoiadas por processos automatizados e repetíveis. A sigla CI refere-se à integração contínua, prática na qual alterações no código são frequentemente integradas, verificadas e validadas por meio de etapas controladas. CD pode indicar entrega contínua ou implantação contínua, dependendo do grau de automação adotado, mas em ambos

os casos remete à capacidade de preparar a aplicação para publicação de modo previsível.

Uma aplicação raramente utiliza os mesmos valores nos ambientes de desenvolvimento, de homologação e de produção. Endereços de serviços, strings de conexão, chaves, políticas de log, origens permitidas, modos de diagnóstico e credenciais precisam variar conforme o ambiente. Ao mesmo tempo, essas diferenças devem ser controladas de forma segura e rastreável.

O empacotamento também merece atenção. Para que uma aplicação seja entregue de modo consistente, é necessário definir quais artefatos representam uma versão pronta para implantação. Esses artefatos podem incluir a aplicação compilada, arquivos da interface, configurações esperadas, scripts de preparação, recursos estáticos e metadados de versão. Quando o empacotamento é claro, torna-se possível saber exatamente o que está sendo publicado.

8.1 Configuração por ambientes, secrets e publicação containerizada

Endereços de serviços externos, strings de conexão, níveis de log, opções de cache, políticas operacionais e chaves de integração variam conforme o ambiente em que a aplicação está sendo executada. O ambiente de desenvolvimento costuma privilegiar diagnóstico, facilidade de execução local e dados controlados; o ambiente de homologação procura reproduzir condições próximas da produção; o ambiente de produção exige configuração segura, menor exposição de detalhes internos e cuidado com disponibilidade. Separar configuração do código permite que a mesma aplicação seja executada em contextos diferentes sem recompilar a solução e sem alterar classes apenas para trocar valores operacionais.

No ASP.NET Core, a configuração é composta por várias fontes. Arquivos como `appsettings.json` e `appsettings.Development.json` fornecem valores estruturados em JSON. Arquivos específicos de ambiente, como `appsettings.Staging.json` e `appsettings.Production.json`, permitem especializar o comportamento da aplicação conforme o valor de ambiente carregado. Variáveis de ambiente podem sobrescrever valores de arquivos e são importantes em servidores e containers. User Secrets permitem armazenar valores sensíveis no ambiente local de desenvolvimento, fora do repositório. O ponto central é que a aplicação consulta uma configuração

unificada, mas essa configuração é montada a partir de fontes diferentes, com precedência e finalidade próprias. Um secret é uma configuração sensível. Senhas, tokens, chaves de API e strings de conexão reais pertencem a essa categoria porque sua exposição pode comprometer dados, acesso ou infraestrutura. Nem toda configuração é secret: o nome público da aplicação, um tempo de expiração de cache ou uma mensagem visual segura podem estar em appsettings.json. Um secret, por outro lado, não deve ser gravado em arquivos versionados, não deve aparecer em prints de tela, não deve ser colocado no Dockerfile e não deve ser copiado para a imagem do container.

Os containers resolvem outro aspecto da preparação operacional: a reprodutibilidade do ambiente de execução. Um Dockerfile descreve como a aplicação é construída, publicada e executada dentro de uma imagem. Isso reduz a dependência de configurações manuais da máquina do desenvolvedor e aproxima a aplicação de um processo de entrega previsível. O container, entretanto, não elimina a necessidade de configuração segura. Ao contrário, ele torna essa disciplina mais importante, pois uma imagem pode ser compartilhada e executada em vários lugares. Por isso, o Dockerfile deve conter instruções de build e runtime, mas não deve conter secrets.

estrutura criada para organizar a configuração começa pela pasta Configuration, no projeto `PetMatch.Web`. O arquivo `PetMatch.Web/Configuration/PetMatchApplicationOptions.cs` representa valores públicos e seguros da aplicação.

```
namespace PetMatch.Web.Configuration;

public sealed class PetMatchApplicationOptions
{
    public const string SectionName = "PetMatch:Application";

    public string Name { get; init; } = "PetMatch";
    public string PublicMessage { get; init; } = "Adoção responsável começa com informação clara.";
}
```

A configuração da integração de dicas de adoção responsável fica em `PetMatch.Web/Configuration/AdoptionTipsOptions.cs`. Essa classe concentra opções do serviço de dicas, incluindo URL, caminho, timeout, tentativas, limite do circuit breaker e nome de cabeçalho. A propriedade `ApiKey` é opcional e deve ser fornecida por `User Secrets` em desenvolvimento ou por variável de ambiente em execução containerizada.

```
namespace PetMatch.Web.Configuration;

public sealed class AdoptionTipsOptions
{
    public const string SectionName = "PetMatch:AdoptionTips";

    public string BaseUrl { get; init; } = "https://localhost";
    public string Path { get; init; } = "/api/dicas/adocao-parceiro";
    public int TimeoutSeconds { get; init; } = 3;
    public int RetryAttempts { get; init; } = 3;
    public int CircuitBreakerFailures { get; init; } = 3;
    public string ApiKeyHeaderName { get; init; } = "X-PetMatch-Tips-Key";
    public string? ApiKey { get; init; }
}
```

O arquivo `PetMatch.Web/appsettings.json` passa a conter apenas valores comuns e seguros. Ele é a base da configuração e pode definir opções que não dependem de credenciais.

194

DESENVOLVIMENTO WEB COM .NET

```
{
  "PetMatch": {
    "Application": {
      "Name": "PetMatch",
      "PublicMessage": "Adoção responsável começa com informação clara."
    },
    "AdoptionTips": {
      "BaseUrl": "https://localhost",
      "Path": "/api/dicas/adocao-parceiro",
      "TimeoutSeconds": 3,
      "RetryAttempts": 3,
      "CircuitBreakerFailures": 3,
      "ApiKeyHeaderName": "X-PetMatch-Tips-Key"
    }
  }
}
```

Esse trecho demonstra a configuração comum da aplicação. Não há senha, token ou chave de API. A seção `PetMatch:Application` contém dados públicos e a seção `PetMatch:AdoptionTips` contém parâmetros operacionais seguros. A propriedade `ApiKey` não aparece nesse arquivo. O arquivo `PetMatch.Web/appsettings.Development.json` mantém valores adequados à execução local, incluindo logs e banco SQLite de desenvolvimento. Ele pode especializar a configuração sem comprometer a segurança.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "PetMatch.Web": "Information"
    }
  },
  "ConnectionStrings": {
    "PetMatchConnection": "Data Source=petmatch-dev.db"
  }
}
```

Os arquivos `PetMatch.Web/appsettings.Staging.json` e `PetMatch.Web/appsettings.Production.json` demonstram a separação de ambientes sem incluir credenciais reais. Eles devem conter placeholders seguros ou valores públicos, deixando secrets para variáveis de ambiente ou mecanismo seguro do provedor.

```
{
  "PetMatch": {
    "Application": {
      "Name": "PetMatch Homologação",

```

Unidade IV

```
    "PublicMessage": "Ambiente de homologação para validação operacional."
  },
  "AdoptionTips": {
    "BaseUrl": "https://homologacao.exemplo-seguro.invalid",
    "Path": "/api/dicas/adocao-parceiro"
  }
}
```

Esse exemplo representa homologação com um endereço ilustrativo e seguro. O valor não é uma credencial. A função do arquivo é demonstrar a variação por ambiente, não fornecer dados reais de operação.

```
{
  "PetMatch": {
    "Application": {
      "Name": "PetMatch",
      "PublicMessage": "Cuidado responsável para adoção de pets."
    },
    "AdoptionTips": {
      "BaseUrl": "https://producao.exemplo-seguro.invalid",
      "Path": "/api/dicas/adocao-parceiro"
    }
  }
}
```

O registro das opções ocorre em `PetMatch.Web/Program.cs`. Esse arquivo preserva a configuração já existente da aplicação, como MVC, Identity, Minimal APIs, Health Checks, observabilidade, rate limiting e output caching, e acrescenta o binding das opções tipadas.

```
builder.Services.Configure<PetMatchApplicationOptions>(
    builder.Configuration.GetSection(PetMatchApplicationOptions.SectionName));
```

```
builder.Services.Configure<AdoptionTipsOptions>(
    builder.Configuration.GetSection(AdoptionTipsOptions.SectionName));
```

```
public sealed class AdoptionTipClient(
    HttpClient httpClient,
    IOptionsMonitor<AdoptionTipsOptions> options,
    ILogger<AdoptionTipClient> logger)
{
    private readonly HttpClient _httpClient = httpClient;
    private readonly IOptionsMonitor<AdoptionTipsOptions> _options = options;
    private readonly ILogger<AdoptionTipClient> _logger = logger;

    public async Task<AdoptionTipResponse> GetTipAsync(CancellationToken
cancellationToken)
    {
        var currentOptions = _options.CurrentValue;

        if (!string.IsNullOrEmpty(currentOptions.ApiKey))
        {
            _httpClient.DefaultRequestHeaders.Remove(currentOptions.
ApiKeyHeaderName);

            _httpClient.DefaultRequestHeaders.Add(
                currentOptions.ApiKeyHeaderName,
                currentOptions.ApiKey);
        }

        // restante da chamada resiliente preservado
        return await BuscarDicaComFallbackAsync(currentOptions,
cancellationToken);
    }
}
```

A tela /Pets continua sendo uma página de usuário. O arquivo `PetMatch.Web/Views/Pets/Index.cshtml` foi ajustado para exibir uma mensagem pública configurável e a faixa de dica de adoção responsável, sem expor ambiente técnico, secrets ou dados internos.

```
@if (!string.IsNullOrEmpty(ViewBag.PublicMessage))
{
    <section class="public-message">
        <p>@ViewBag.PublicMessage</p>
    </section>
}
```

O projeto também recebeu `UserSecretsId` em `PetMatch.Web.csproj`, habilitando o fluxo `Manage User Secrets` pelo Visual Studio 2026. O objetivo é permitir que o desenvolvedor armazene localmente um valor fictício para `PetMatch:AdoptionTips:ApiKey`, sem colocar esse valor no repositório.

```
<PropertyGroup>
  <UserSecretsId>petmatch-web-development-secrets</UserSecretsId>
</PropertyGroup>
```

DESENVOLVIMENTO WEB COM .NET

Esse identificador associa o projeto a um arquivo local de secrets mantido fora da pasta do projeto. O valor usado no livro é fictício. A prática correta é entender o mecanismo e não copiar credenciais reais. A containerização foi preparada com um `Dockerfile` em `PetMatch.Web/Dockerfile`. Esse arquivo descreve como restaurar, publicar e executar a aplicação ASP.NET Core dentro de uma imagem compatível com .NET 10.

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
WORKDIR /app
EXPOSE 8080
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ["PetMatch.Web.csproj", "."]
RUN dotnet restore "PetMatch.Web.csproj"
COPY .
RUN dotnet publish "PetMatch.Web.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=build /app/publish.
ENTRYPOINT ["dotnet", "PetMatch.Web.dll"]
```

8.2 Deploy integrado (API servindo estáticos do React) ou pipelines separados (API + front-end)

No deploy integrado, a aplicação ASP.NET Core publica a API e também serve os arquivos estáticos gerados pelo front-end React. O React, nesse modelo, deixa de ser executado por um servidor de desenvolvimento como Vite e passa a existir como um conjunto de arquivos estáticos: HTML, CSS, JavaScript e assets. Esses arquivos são colocados em uma pasta pública do projeto ASP.NET Core, e o próprio PetMatch.Web passa a entregá-los ao navegador. A vantagem principal é a simplicidade operacional: uma aplicação, um host, um domínio, uma configuração central e menor exposição a problemas de CORS. A desvantagem é o acoplamento entre os ciclos de entrega: uma alteração no front-end pode exigir a publicação do pacote completo e uma falha no pipeline integrado pode bloquear a entrega do conjunto.

No deploy separado, API e front-end seguem caminhos independentes. A API ASP.NET Core pode ser publicada como aplicação web ou container, enquanto o React pode ser hospedado como site estático em um serviço especializado ou CDN. Essa abordagem favorece equipes separadas, cache agressivo de arquivos estáticos, escalabilidade independente e rollback isolado. Em contrapartida, exige cuidado maior com URLs da API, CORS, cookies, autenticação, variáveis de ambiente do front-end, compatibilidade entre versões e coordenação dos pipelines. Assim, a decisão não é uma escolha puramente tecnológica; é uma decisão de entrega, operação e governança.

Para representar essa decisão no código, foi criada a classe `FrontendDeploymentOptions`, no projeto `PetMatch.Web`, em `Configuration/FrontendDeploymentOptions.cs`.

```

namespace PetMatch.Web.Configuration;

public sealed class FrontendDeploymentOptions
{
    public const string SectionName = "PetMatch:FrontendDeployment";

    public string Mode { get; init; } = "Integrated";
    public string IntegratedPath { get; init; } = "/app";
    public string StaticFilesPath { get; init; } = "wwwroot/app";
    public string ApiBasePath { get; init; } = "/api";
    public string IndexFile { get; init; } = "index.html";
}

```

Os arquivos `appsettings.json`, `appsettings.Development.json`, `appsettings.Staging.json` e `appsettings.Production.json` foram ajustados para incluir opções seguras de entrega.

```

{
  "PetMatch": {
    "FrontendDeployment": {
      "Mode": "Integrated",
      "IntegratedPath": "/app",
      "StaticFilesPath": "wwwroot/app",
      "ApiBasePath": "/api",
      "IndexFile": "index.html"
    }
  }
}

```

Essa configuração informa que, quando os arquivos do React forem integrados, eles serão servidos a partir de `/app`. O caminho `/api` permanece reservado às Minimal APIs e `/Pets` continua sendo a área MVC principal.

A pasta `PetMatch.Web/wwwroot/app` foi criada para receber os artefatos estáticos do React quando a estratégia integrada for usada. O arquivo `.gitkeep` mantém a pasta no repositório mesmo quando ela ainda não possui `index.html`, CSS ou JavaScript.

O mapeamento no `Program.cs`, no projeto `PetMatch.Web`, foi ajustado para servir a SPA integrada apenas quando o arquivo inicial existir. O trecho essencial preserva a aplicação MVC e a API, restringindo o fallback da SPA ao caminho `/app`.

```
var frontendOptions = app.Services
    .GetRequiredService<IOptions<FrontendDeploymentOptions>>()
    .Value;

var integratedIndexPath = Path.Combine(
    app.Environment.WebRootPath,
    "app",
    frontendOptions.IndexFile);

if (File.Exists(integratedIndexPath))
{
    app.MapFallbackToFile(
        $"{frontendOptions.IntegratedPath}/{*path:nonfile}",
        $"app/{frontendOptions.IndexFile}");
}
```

O arquivo `PetMatch.Web.csproj` recebeu o alvo opcional `CopyIntegratedFrontendAssets`, desligado por padrão. A finalidade é documentar o ponto de integração entre o build do React e a publicação do ASP.NET Core sem obrigar o leitor a executar o front-end na prática local. O alvo só deve ser usado quando houver artefatos reais do React prontos para cópia.

```
<Target Name="CopyIntegratedFrontendAssets"
    BeforeTargets="ComputeFilesToPublish"
    Condition="'$(UseIntegratedFrontendAssets)' == 'true'">
    <ItemGroup>
        <IntegratedFrontendFiles Include="..\PetMatch.Frontend\dist\**\*.*)" />
    </ItemGroup>

    <Copy SourceFiles="@ (IntegratedFrontendFiles)"
        DestinationFolder="wwwroot\app\%(RecursiveDir)" />
</Target>
```

Também foram criados três workflows em `.github/workflows`: `petmatch-integrated.yml`, `petmatch-api.yml` e `petmatch-frontend.yml`. O primeiro representa a entrega integrada do `PetMatch.Web`. O segundo representa um pipeline separado para a API. O terceiro representa um pipeline separado para o front-end. Esses arquivos são modelos de CI/CD e não precisam ser executados localmente pelo leitor. O ponto mais importante é que eles não contêm secrets reais; qualquer credencial necessária deve aparecer apenas como referência simbólica a secrets do ambiente de CI.

```
name: PetMatch Integrated

on:
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
```

```
steps:
  - name: Checkout
    uses: actions/checkout@v4
```